# Rail Simulator Audio Control and Blueprints

Release version 1.0 (2007-11-07)


**1 INTRODUCTION ..... 3**

1.1 Audio Controls ..... 6

*1.1.1 Volume ..... 6*
*1.1.2 Pitch ..... 7*
*1.1.3 A Generic Audio Control ..... 8*
*1.1.4 Controlled Loop Conditions ..... 14*
*1.1.5 Triggered Oneshots trigger types ..... 14*
*1.1.6 Value is changing gate time (seconds) ..... 14*

1.2 Ambient Sounds ..... 15

*1.2.1 Ambient Loop ..... 16*
*1.2.2 Ambient Random Oneshots ..... 20*

1.3 Road Sounds ..... 22

**2 RAILNETWORK SOUNDS ..... 24**

2.1 Bogies ..... 24
2.2 Coupling ..... 25
2.3 TrackBedRumble ..... 26
2.4 Derailing ..... 30
2.5 Junctions ..... 31
2.6 Reverb ..... 31
2.7 Platforms ..... 32
2.8 Signals ..... 32
2.9 TransferPoints ..... 33
2.10 Traversers + Turntables ..... 34

**3 LOCOMOTIVES ..... 34**


©Rail Simulator Developments 2007

3.1 Diesel Engines ..... 35
3.2 Electric Engines ..... 36
3.3 Steam Engines ..... 36
3.4 Wagons ..... 38
*3.4.1 Wagons ..... 38*
*3.4.2 Passenger Carriage Occlusion ..... 39*
3.5 Cabs ..... 39
*3.5.1 Cabs ..... 39*
*3.5.2 Horns ..... 39*
*3.5.3 Occlusion ..... 40*
**4 ANIMATED SCENERY ..... 41**
**5 WEATHER ..... 42**

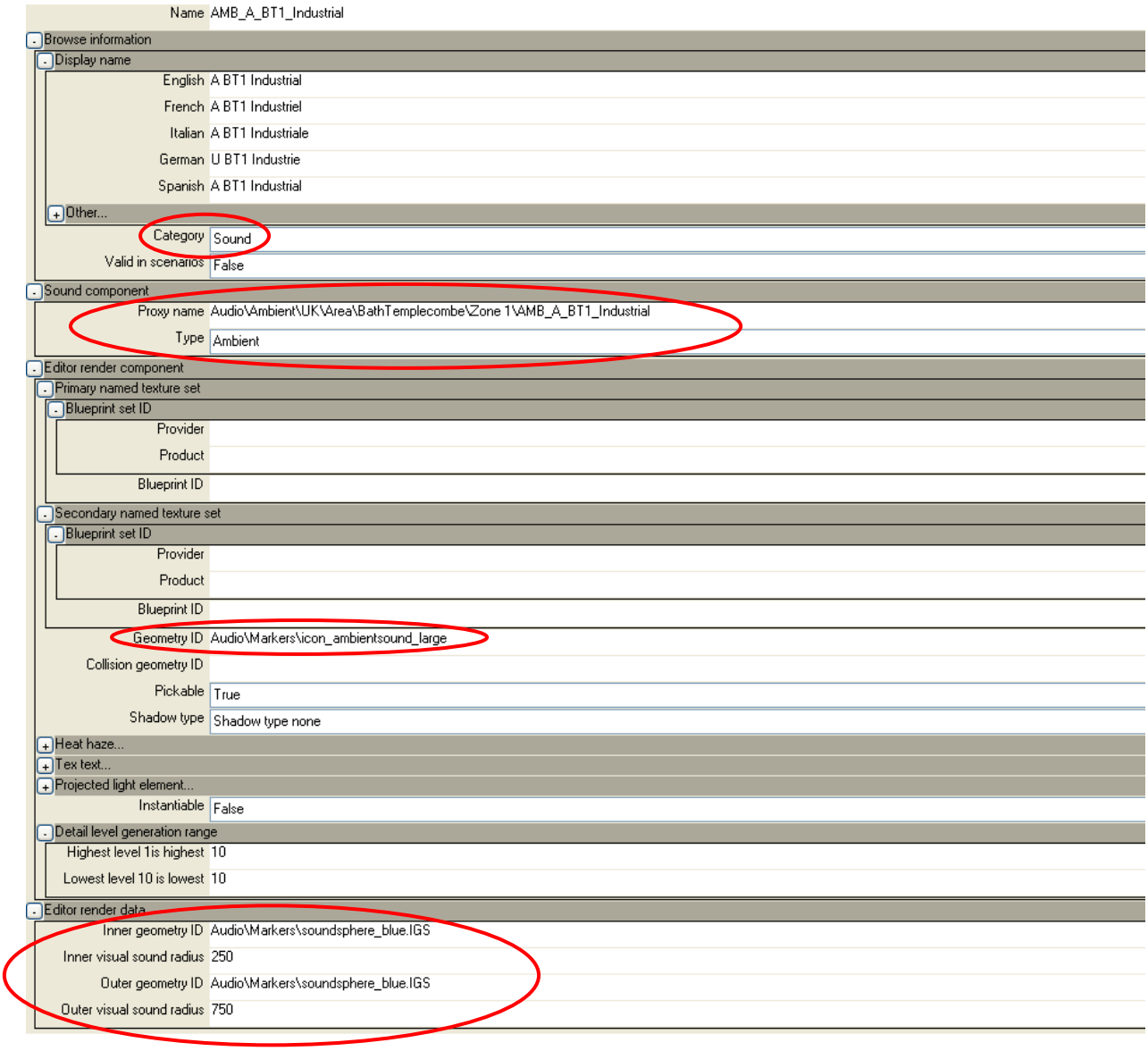
© Copyright Rail Simulator Developments 2007, all rights reserved

# 1 Introduction

*TIP: When developing sound for Railsim it is highly recommended to use an EAX enabled soundcard, Without an EAX card you cannot use the Reverb or Occlusion parameters. Even if you deem these effects unnecessary you will not be able to simulate how users* ***with*** *an EAX card will experience your work.*

To implement audio in Rail Simulator you should start by reading the document on the blueprint editor. To create an audio Blueprint right click on a folder in the Blueprint editor, select 'New Blueprint' and then select 'Sound Blueprint'

A basic Audio blueprint will look something like this.




A description of the Sound Blueprint unique features:

© Copyright Rail Simulator Developments 2007, all rights reserved

**Category:** Should always be set to **Sound** for anything placeable in the world, otherwise for train engine sounds or things which are directly attached to an object which you do not want to user to place in the world select **Exclude from browser list.**

**Sound Component**: This is a link to the relevant Audio Control XML file (where all the hard work is done, see section 1.1 below for more details).

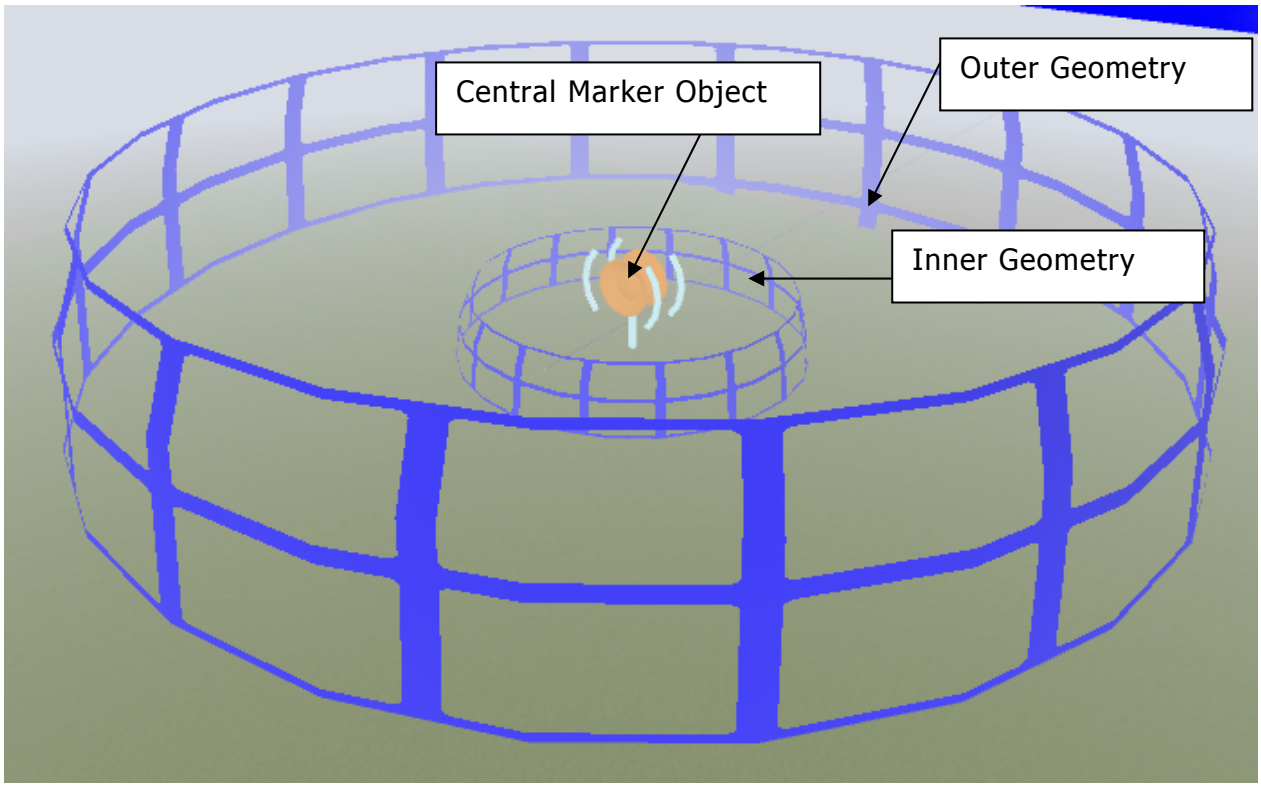
**Type**: **It is vital this is set correctly**:

> **Ambient**: Any sound which is not part of a Consist or the Rail Network. E.g. the sound of churchbells placed in the world or attached to a church model, environmental sounds, road sounds, factory sounds, birdsong etc. All sounds set to this type respond to the front end audio options volume slider 'Ambient Volume'.
>
> **Cab:** For the sounds which you only hear when inside a train cab. Levers, buttons, AWS, etc. These sounds are **always** disabled when outside the cab view. For Locos with Open backs you may want to include some sounds in a **RailVehicle** type instead. E.g. for loud internal sounds which you expect to also hear externally.
>
> **Passenger:** The same as Cab but for sounds to be heard only inside Passenger carriage internal views.
>
> **RailVehicle:** All rolling stock sounds other than internal cab sounds. Engine, brakes, horns etc.

**Geometry ID:** (The instance ringed in red) This is the central marker object that represents the audio positioning in the game editor or depot. Accepts .IGS file references.

**Editor Render Data:** These control the sizes of the attenuation visual identifiers which you can see in the editor when you place one of the stock ambient sounds, these are set by eye and ear, they are not directly related to the actual audio attenuation. Radius in metres which scales up a spherical IGS object 1 metre in diameter. i.e. A BT1 Industrial:

© Copyright Rail Simulator Developments 2007, all rights reserved




© Copyright Rail Simulator Developments 2007, all rights reserved

## 1.1 Audio Controls

The Audio Controls which are referenced in the sound blueprint **Sound Component** are where all the actual sound work is carried out.

There are four types which we recommend you use:

**Generic Audio Control**
**Ambient Audio Control**
**Steam Audio Control**
**UI Audio Control**

(Any others in the list are simply Generic Audio Controls with a different name, it is best to just use the Generic version to avoid any confusion)

All these Audio Controls are essentially variants on the Generic Audio Control with some specific features described later in the document. All the information below on the **Generic Audio Control** is relevant to the other three types. You create a new audio control by right clicking on a folder in the blueprint editor, selecting **New Audio Control** and then picking a type.

There are two key RailSim sound variables to know about:

### 1.1.1 Volume

Controls how loud a sound plays back.
RailSimulator uses a linear float for Volume ranging from 0 -> 1, a good rule of thumb is that halving a volume value will decrease it by -6dB. So:

| RailSim Volume (linear) | Decibel (dB) (logarithmic) |
|---|---|
| 1 | 0dB (full Volume) |
| 0.5 | -6dB |
| 0.25 | -12dB |
| 0.125 | -24dB |
| 0.015625 | -192dB |
| 0 | ∞ (completely silent) |

It is common to need to use very small fractions ( between 0 -> 0.1) to carefully balance quieter sounds.

© Copyright Rail Simulator Developments 2007, all rights reserved

### 1.1.2 Pitch

Controls the speed or pitch of a sample playback. With a range from 0 -> 2

| RailSim Pitch (linear) | Semitones |
|---|---|
| 2 | +12 (a whole octave up, double speed - > Half the length) |
| 1.5 | +7 ~1.9 cents |
| 1.25992 | + 4 |
| 1 | Normal playback speed |
| 0.5 | -12 (a whole octave down, half speed -> Double the length) |
| 0 | Zero speed, not recommended! |

As you can see, the relationship to musical semitones are not intuitive values so it is best to just set these by ear unless you have experience in this area. Be aware that the limit to pitching sounds up is based on the upper limit of sample frequency your soundcard will play back. If your card will not play samples higher than 44.1khz for example then trying to pitch a 22.05khz sample higher than '2' will result in no extra pitchshifting. All soundcards will play back at least 44.1khz these days, with a great deal of them handling 48khz or higher, but if you want your audience to have the same experience you should not be trying to play anything back higher than 44-48khz after pitching.

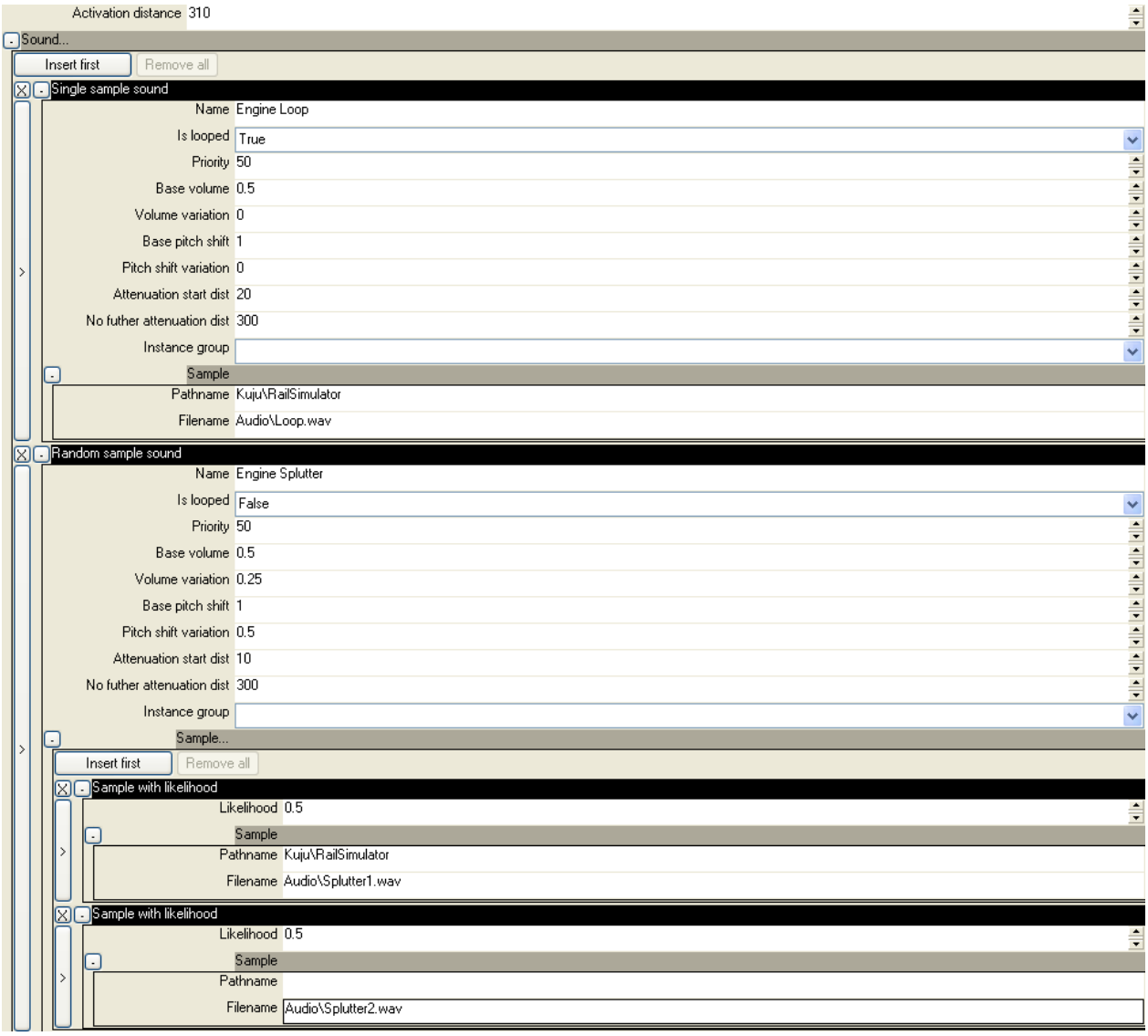
© Copyright Rail Simulator Developments 2007, all rights reserved

### 1.1.3 A Generic Audio Control

This Audio Control is an example of a sound you could place on a static 'eyecandy' train. It has a continuous looping engine sound and random engine splutters.

#### 1.1.3.1 Section 1 – Sound

This is where you reference your raw sound files and give some basic playback info. You should always use **Mono** sounds except in the case of **Ambient Areas**, described later.




© Copyright Rail Simulator Developments 2007, all rights reserved

**Activation Distance:** 310 Metres (notice this is slightly larger than the **No further Attenuation Distance** below)

So we have a **Single Sample Sound:**

**Name:** Engine Loop
**Is Looped:** True (True for a looped sound, False for a 'oneshot' sound)
**Priority: 50** (A value from 1-100, if the soundcard is being asked to play too many channels it will start ignoring the lower priority rated sounds first)
**Base Volume:** 0.5 (Equivalent to -6db)
**Volume Variation:** 0 (The sound always plays at it's base volume)
**Base Pitch Shift:** 1 ('1' Plays at normal speed)
**Pitch Shift Variation:** 0 (The sound always plays at it's base pitch)
**Attenuation Start Distance:** 20 Metres (As you move away from the sound it will start getting quieter 20 metres from it's position)
**No further Attenuation Distance:** 300 Metres (As you move away from the sound it will stop getting any quieter after 300 Metres)
**Pathname:** The product pathname
**Filename:** The soundfile location. (Mono Wav files, 16bit 11.025 -> 44.1 khz recommended)

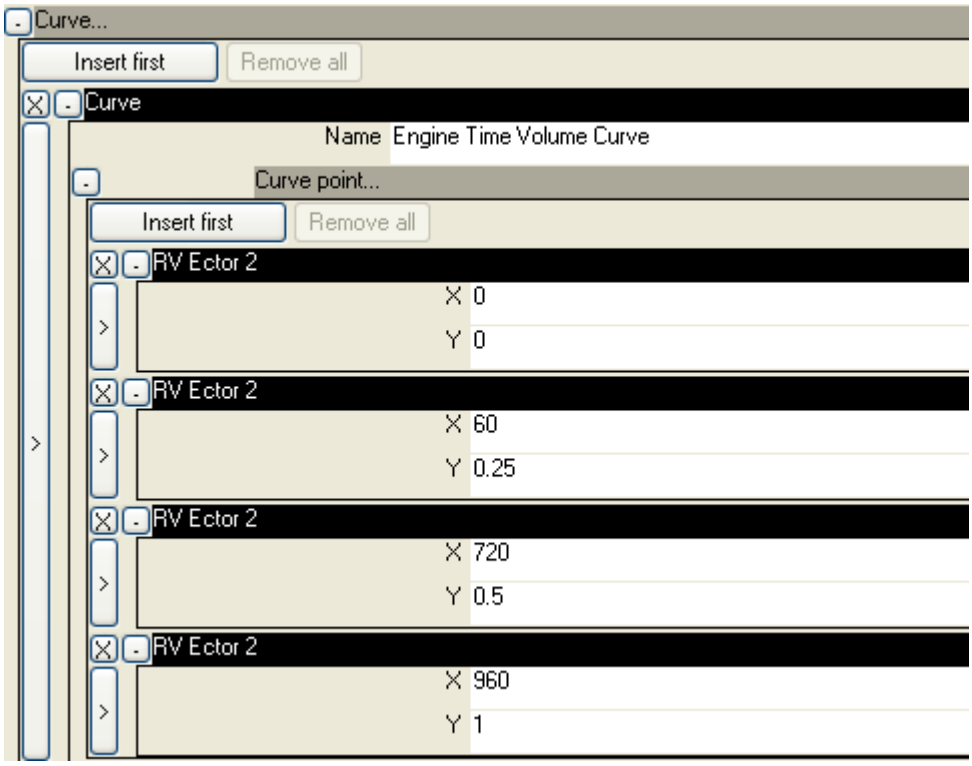
And we also have a **Random Sample Sound:**

**Name:** Engine Splutter
**Is Looped:** False (the sound plays a single time for every trigger)
**Priority: 50**
**Base Volume:** 0.5 (Equivalent to -6db)
**Volume Variation:** 0.25 (Everytime the sound is triggered it will add a random value between -0.25 and 0.25 to the base volume, so this sound could actually end up being played at any volume between 0.25 -> 0.75)
**Base Pitch Shift:** 1 ('1' Plays at normal speed)
**Pitch Shift Variation:** 0.5 (Everytime the sound is triggered it will add a random value between -0.25 and 0.5 to the base pitch shift, so this sound could actually end up being played at any pitch between 0.75 -> 1.5. The minus value is only -0.25 due to octave relationship described in the pitch section above)
**Attenuation Start Distance:** 10 Metres (As you move away from the sound it will start getting quieter 10 metres from it's position, so the splutters will disappear first)
**No further Attenuation Distance:** 300 Metres (As you move away from the sound it will stop getting any quieter after 300 Metres)
**Liklihood:** 0.5 (Random sample sounds will play one of the *sample with likelihoods* every time it is triggered, as both Splutter1.wav and Splutter2.wav has the same likelihood they are both equally as likely to be played. If Splutter 2.wav had a likelihood value of 0.25 then on average it would only play half as many times as Splutter1.wav. These liklihoods do not have to all add up to 1, they are completely relative within the specific Random Sample Sound.)
**Pathname:** The product pathname
**Filename:** The soundfile location. (Mono Wav files, 16bit 11.025 -> 44.1 khz recommended)

© Copyright Rail Simulator Developments 2007, all rights reserved

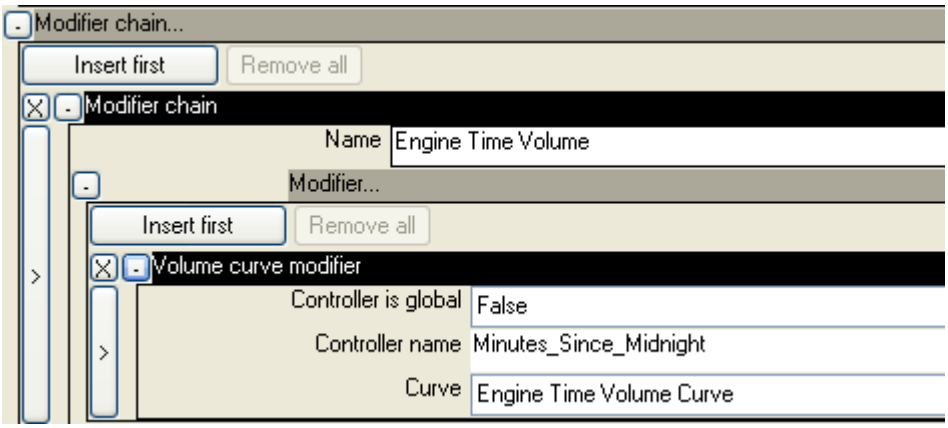
### 1.1.3.2 Section 2 – Curve

Let's add a Curve which changes the volume of the engine loop based on… the time of day!have a controller available called **Minutes_Since_Midnight** which we will use.

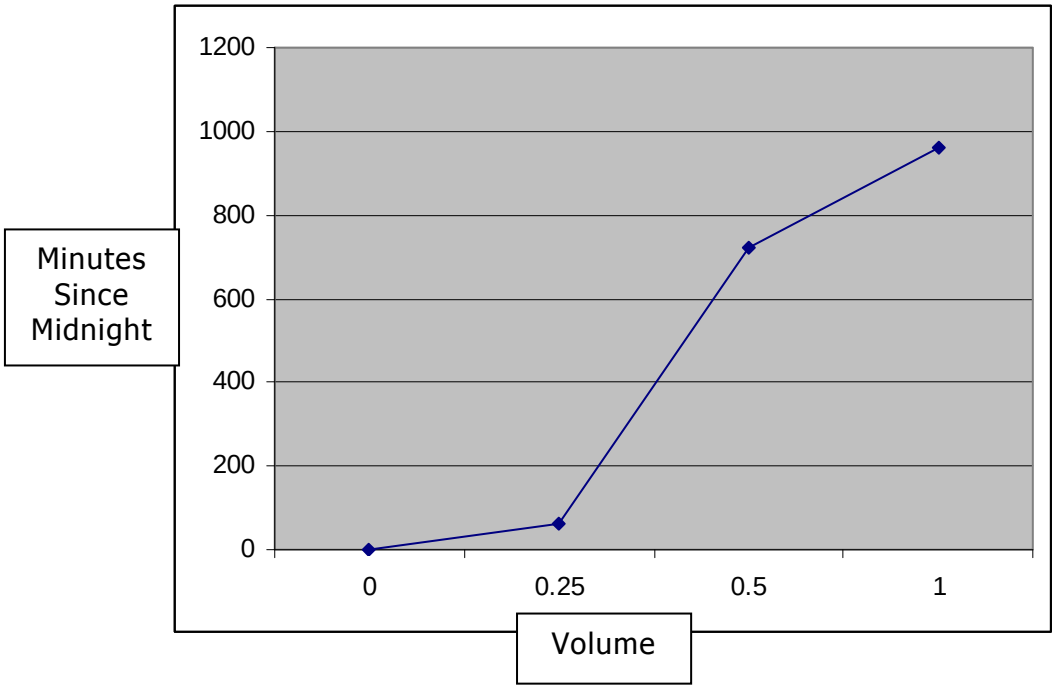



© Copyright Rail Simulator Developments 2007, all rights reserved

### 1.1.3.3 Section 3 – Modifier Chain




This is where we tell the curve what to act upon to produce the following graph:




So we have created a **Volume Curve Modifier** which means the X-axis of our graph is volume, and in the controller name we have typed **Minutes_Since_Midnight** which becomes our Y-Axis and we reference our earlier created curve **Engine Time Volume Curve.**

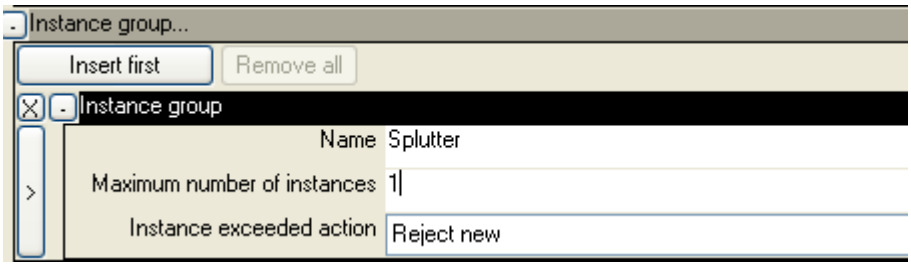
So once this modifier chain is applied to our sound (in the next section) the volume of that sound will change depending on the time of day. At midnight (0 minutes) it will play at zero volume, at 1.00am (60 minutes) it will play at 0.25 volume, and at 4pm (960 minutes) it will play at Full volume ('1' volume).

© Copyright Rail Simulator Developments 2007, all rights reserved

Within a modifier chain you can have as many volume curves as you like and they all interact together, but they can never push the volume value above 1. **Pitch Curve Modifiers** work in exactly the same way, but again even with multiple interacting pitch curves the value can never go outside the limits of the pitch parameter itself (0 -> 2, as defined in section 1.1 above)

### 1.1.3.4 Section 4 – Instance Group

Instance groups are a useful tool for stopping certain sounds from ever playing at the same time.




We will apply this instance group to our engine splutter sound.

**Maximum number of instances:** 1 (any sound triggers with this instance group applied will only ever produce 1 channel of sound at any time, regardless of the number of triggers it is applied to)

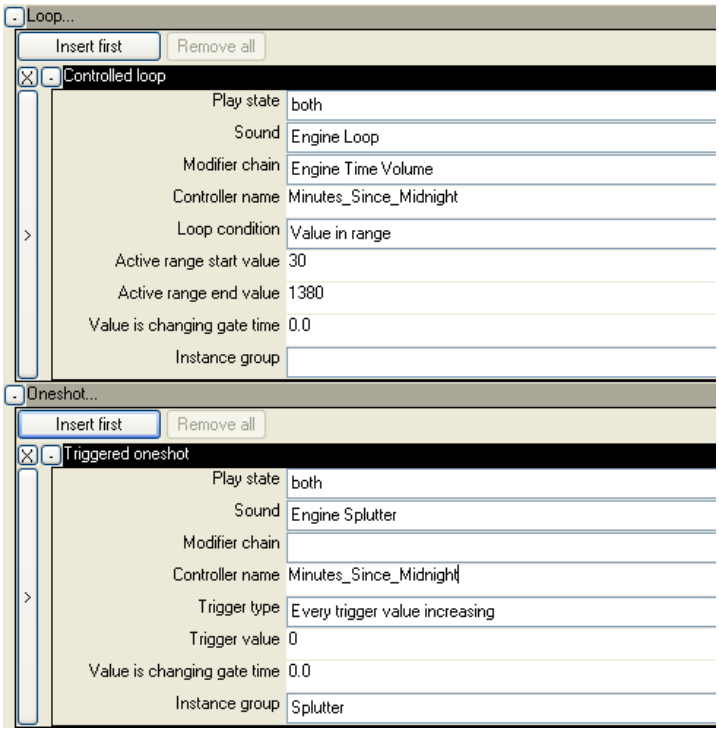
**Instance Exceed Action:**

> **Reject New** – No further sounds from this instance group will play until the current sound has finished or reached the end of the file, I.e. all new trigger requests are rejected.
>
> **Stop Oldest –** If a new trigger request is made then a new sound will play immediately and the oldest playing sound in the instance group will stop immediately. (Becomes more useful with maximum number of instances larger than 1)
>
> **Stop Newest -** If a new trigger request is made then a new sound will play immediately and the newest playing sound in the instance group will stop immediately. (Becomes more useful with maximum number of instances larger than 1)

© Copyright Rail Simulator Developments 2007, all rights reserved

### 1.1.3.5 Section 5 & 6 – Loops and Oneshots

This is where we tie everything together in the audio control. We tell the game when and how to play sounds.




In the **Loop** section we have created a **Controlled Loop.**

**Play State:**

**Both –** You can hear the sound wherever you are in the game.
**Inside** – You can only hear the sound inside an engine cab
**Outside** – You can hear the sound everywhere except inside a engine cab.

**Sound:** Engine Loop (The sound we originally created back in section 1)
**Modifier Chain:** Engine Time Volume (The modifier we created back in section 3)

**Controller name:** Minutes_Since_Midnight
**Loop Condition:** Value in range
**Active Range start value:** 30
**Active Range end value:** 1380

These four parameters decide when the loop starts and stops playing. In this case it will start playing at 00.30am and stop playing at 11pm (1380 minutes since midnight). Details on other loop conditions below.

In the **Oneshot** section we have created a **Triggered Oneshot**. It is similar to the **Controlled Loop** with the following differences.

© Copyright Rail Simulator Developments 2007, all rights reserved

**Trigger Type:** Every trigger value increasing (This means that everytime the **Minutes_Since_Midnight** value increments a oneshot sound is triggered, so it will trigger a sound every minute)

**Instance Group:** Were you to try and trigger sounds longer than 1 minute in length this would stop 2 of them from ever playing at the same time. Instance groups can also be assigned right at the top of the Audio Control in the **Sounds** section, but it makes no difference and is easier to maintain if you assign them down in the Controlled Loop and Oneshots section.

### 1.1.4 Controlled Loop Conditions

**Value in range** – Loop will play whenever the Controller value is within the active range start and end values.
**Value is changing up** – Loop will play whenever the Controller value is changing upwards. Active range ignored.
**Value is changing** - Loop will play whenever the Controller value is changing upwards or downwards. Active range ignored.
**Value is changing down** - Loop will play whenever the Controller value is changing downwards. Active range ignored.

### 1.1.5 Triggered Oneshots trigger types

**Value increase past** – Trigger a oneshot when controller value increased past trigger value.
**Value decrease past –** Trigger a oneshot when controller value decreases past trigger value.
**Value moves past** – Trigger a oneshot when controller value increases or decreases past trigger value.
**Value is changing up** – Trigger a oneshot when controller value is changing upwards.
**Value is changing down** – Trigger a oneshot when controller value is changing downwards.
**Value is changing** – Trigger a oneshot when controller value is changing upwards or downwards.
**Every trigger value increasing** – Trigger a oneshot every time the controller value increments by the trigger value.
**Every trigger value decreasing** – Trigger a oneshot every time the controller value decrements by the trigger value.
**Every trigger value moves past** - Trigger a oneshot every time the controller value increments or decrements by the trigger value.

### 1.1.6 Value is changing gate time (seconds)

It is vital to use this setting when triggering a sounds from a value which often fluctuates or changes unpredictably (i.e. some simulation parameters). If this is set to '10' then no matter how much the controller value changes and is valid to trigger a sound, nothing will happen until 10 seconds has passed from the last trigger. This can be used to stop loops juddering on and off when based on a cab lever moving, or can be used to stop hundreds of oneshots being triggered when triggering from a simulation controller value that changes every frame.

© Copyright Rail Simulator Developments 2007, all rights reserved

## 1.2 Ambient Sounds

Ambient Sounds have a unique Audio Control.

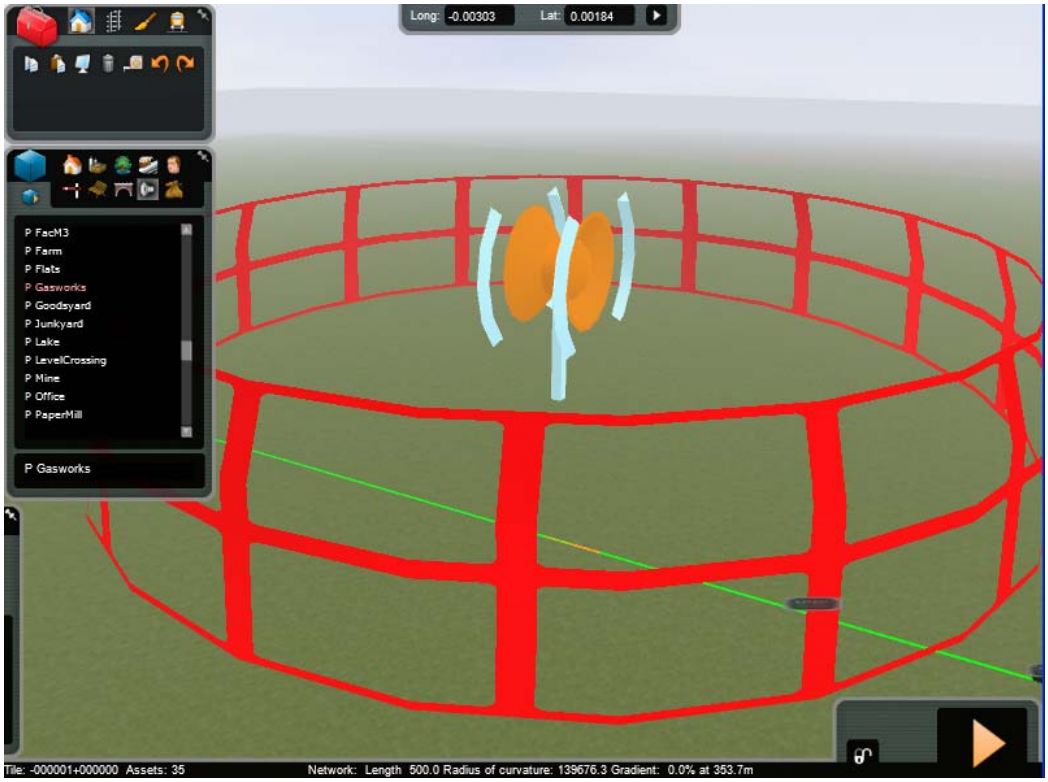
Right click on a folder, select **New Audio Control** and select **Ambient Audio Control**.

The First 4 sections work identically as above, **Sound**, **Curves**, **Modifier Chains** and **Instance Groups**. But the **Loop** and **Oneshot** sections feature different controls.

There are two types of Ambient Sounds:

**Ambient Area** – Stereo Sounds, long loops and random oneshots, background ambience e.g. birdsong or city rumble. These sounds are 'everywhere' when you are inside a zone and do not pan in 3D space.

**Point Sound –** Mono Sounds, shorter loops and random oneshots, specific sounds of objects in the world e.g. Church Bells or a Factory. These sounds exist in 3D so they will pan in the stereo field and also have the **Doppler** effect automatically applied. Good for things at the trackside which will give you a sense of speed when you pass in a train.



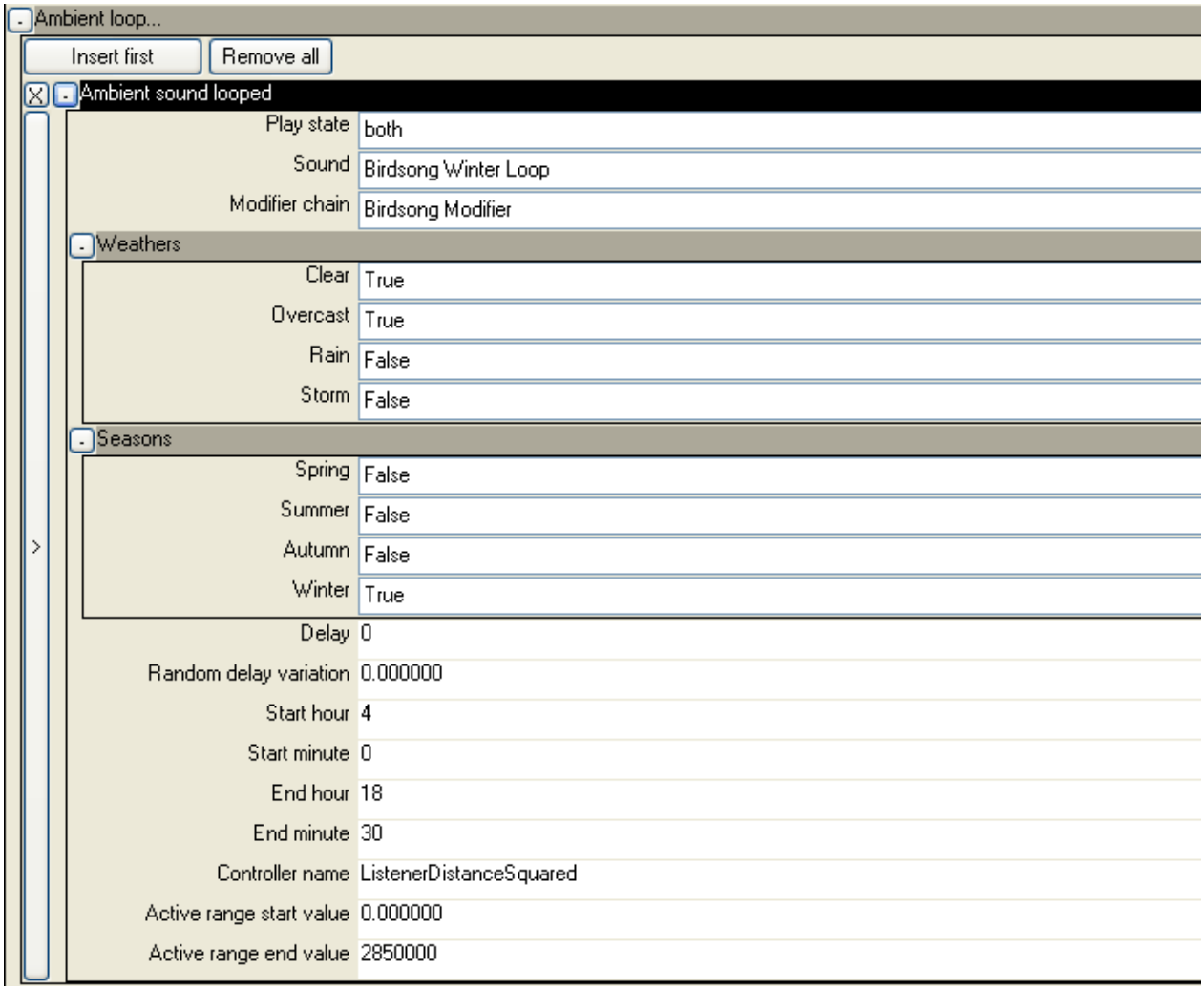

© Copyright Rail Simulator Developments 2007, all rights reserved

### 1.2.1 Ambient Loop

This is **Ambient Audio Control** version of a loop, it can be used for both Ambient Areas and Point Sounds but for this example lets show it's use for an **Ambient Area** as there is a difference in implementation between stereo and mono sounds.

E.g. a Birdsong background loop

First off set an activation Distance radius of 800, which will roughly give you an ambient area of 1600 Metres in diameter.




© Copyright Rail Simulator Developments 2007, all rights reserved

There are some similar features we recognise, **Play state, Sound, Modifier Chain** etc. **Weathers** and **Seasons** simply defines in which weather and season states this loop will play, in this case the birds won't sing when the weather is Rain or Storm (note that this means the birds will just stop dead rather than fade in and out, that can be achieved using the special weather controllers discussed in Section 5 - Weather.) and will only sing in Winter.

**Delay** and **Random delay variation** are not applicable to loops so will be left at Zero (these are used for ambient random oneshots described in a bit).

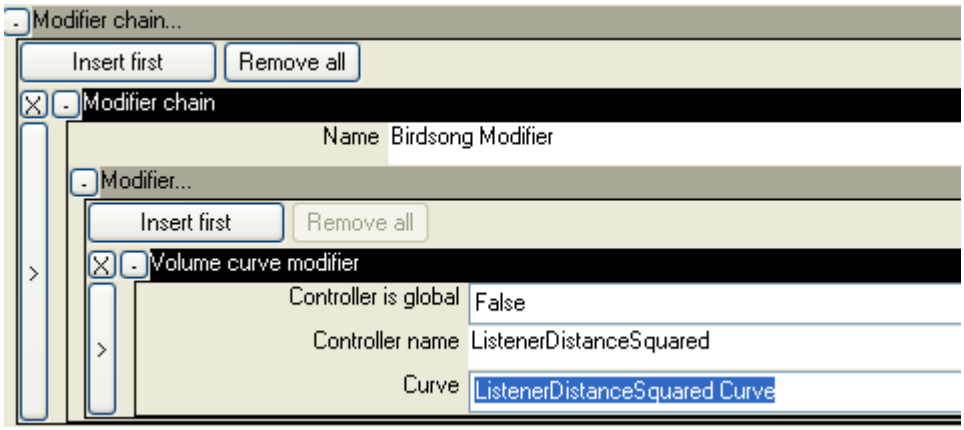
**Start Hour**, **Start Minute**, **End Hour**, **End Minute** are pretty self evident, this loop will start playing at 4.00am and finish playing at 6.30pm. Note that you cannot have a start time which is later than the End Time, for example if you want to have a sound start playing at 9pm and finish at 4am. To achieve this you need to create two separate Ambient Loops with identical settings except one plays from 21.00-24.00 and the other plays from 0.00-4.00.

Now remember this is an example for a Stereo **Ambient Area**, in the Rail Sim audio system Stereo Sounds are treated a little differently. They do not use the Attenuation start and end values as set up in the sound section of the Audio Control. This means we need to define ourselves how the sound drops in volume as you move away from the centre marker. This can be done with the Controller **ListenerDistanceSquared**. This is a squared value, so to specify a curve point or Active range value you need to square the value required.

e.g.

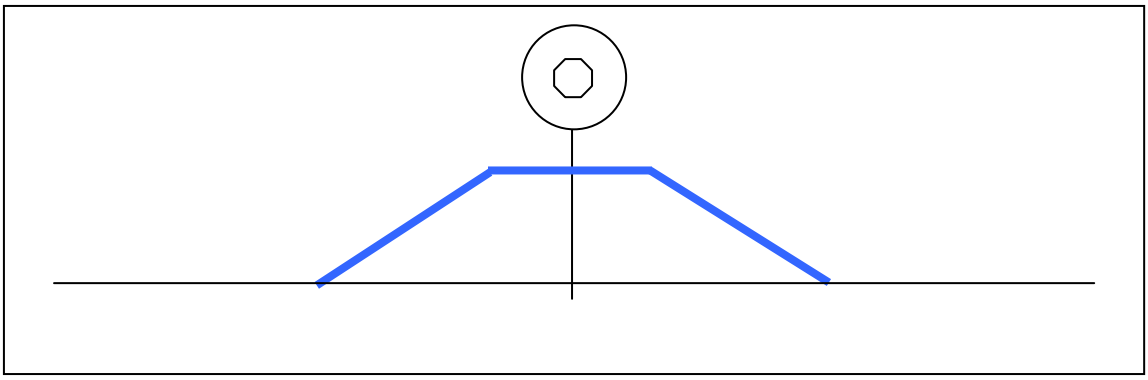
| Required distance (m) | ListenerDistanceSquared (m) |
|---|---|
| 0 | 0 |
| 10 | 100 |
| 53 | 2809 |

So in our example we can see that the sound will have an active range of 0 – 2850000, which in metres (use Square root!) is about 0 -> 1688. So our sounds starts playing when we are 1688 metres or roughly 1.5km away. However this is not controlling the volume as we move away from the centre point. However we can create a modifier chain which works from **ListenerDistanceSquared** and uses a curve to make the sound get quieter as we move away.

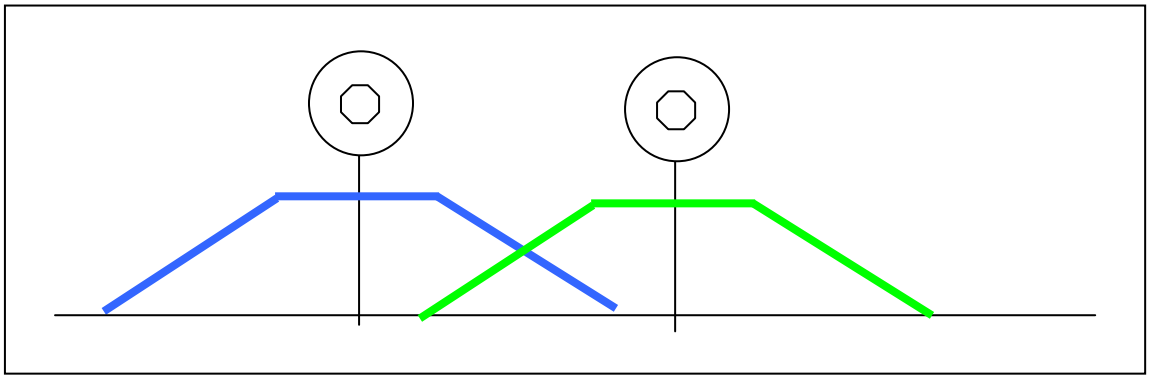



© Copyright Rail Simulator Developments 2007, all rights reserved

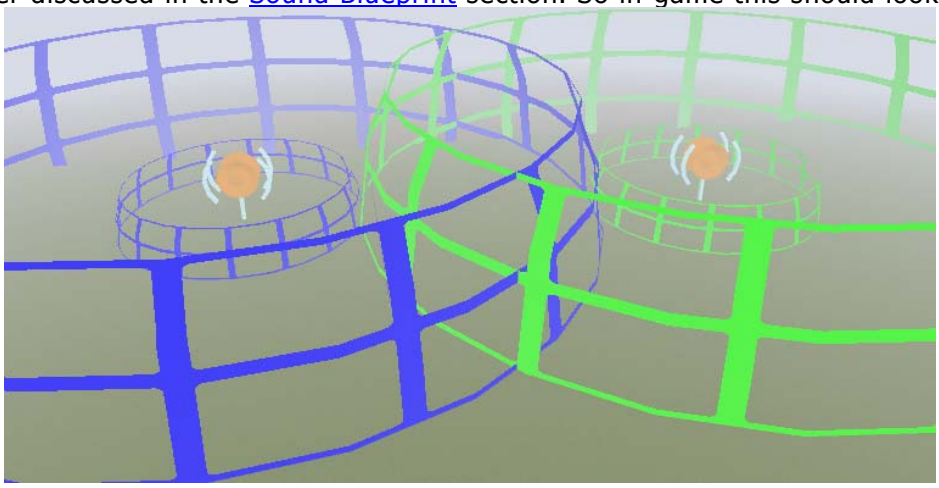
It is a good idea with **Ambient Area** sounds to use curves which nicely interlock with other Ambient Areas to create a nice smooth fade between one area to another, for example when you move from an industrial to a rural zone. You also want the sound to sustain at an equal volume at a certain radius around the centre point. So basically this shape:



Where the blue line shows the volume level as you move towards the ambient sound, in the middle around the sound it stays at constant volume within a certain radius. As you move away the volume of the sound reduces to zero. This allows you to smoothly interlock **ambient areas** like so:



As you move from the Blue Sound to the Green sound you will have a smooth fade between them. This is what is graphically represented by the outer and inner geometry rings around the marker discussed in the Sound Blueprint section. So in-game this should look like this:



© Copyright Rail Simulator Developments 2007, all rights reserved

Now we need to actually define the curve specified in the image of the **modifier chain** on the previous page, which I called **ListenerDistanceSquared Curve**

I suggest you use the following values. With these values you **must** use an **activation distance** of 800 as shown previously, and set the **Active range start and end values** in the **ambient loop** as 0 -> 2850000 as also described on previous pages. Then in your **Sound Blueprint** you can use an **Inner Visual Sound Radius** of 250 and an **Outer Visual Sound Radius** of 750 and you will have an Ambient sounds which looks like the ones in the shot above in the game.

So the curve values are:

| X - ListenerDistanceSquared | Y - Volume |
|---|---|
| 62500 | 1 |
| 75625 | 0.988553 |
| 90000 | 0.977237 |
| 105625 | 0.944061 |
| 122500 | 0.891251 |
| 140625 | 0.841395 |
| 160000 | 0.785236 |
| 180625 | 0.716143 |
| 202500 | 0.645654 |
| 225625 | 0.57544 |
| 250000 | 0.501187 |
| 275625 | 0.42658 |
| 302500 | 0.350752 |
| 330625 | 0.281838 |
| 360000 | 0.216272 |
| 390625 | 0.154882 |
| 422500 | 0.102329 |
| 455625 | 0.058884 |
| 490000 | 0.025704 |
| 525625 | 0.004677 |
| 562500 | 0 |

So from 0 -> 62500 **ListenerDistanceSquared** (or within the **Inner Visual sound Radius** of 250 metres) the volume will be 1, then from 62500 -> 562500 **ListenerDistanceSquared** we see the volume fade from 1 -> 0 (to the **Outer Visual Sound Radius** of 750 metres). **TIP:** X values should always be entered in **Ascending** order.

These curve values use an Equal power fade, which means that at the point when you are equally distant from two ambient areas you maintain a constant volume, if you used a linear fade then the sound would dip dramatically in the middle of the two ambient areas.

You'll notice is no value for 0 **ListenerDistanceSquared,** it is always assumed that the first Y value in a curve will be applied to all X values below the first X value. So the data above would be identical to a data set which starts like the one below:

© Copyright Rail Simulator Developments 2007, all rights reserved

| X - ListenerDistanceSquared | Y - Volume |
|---|---|
| 0 | 1 |
| 62500 | 1 |
| 75625 | 0.988553 |
| Etc. | |

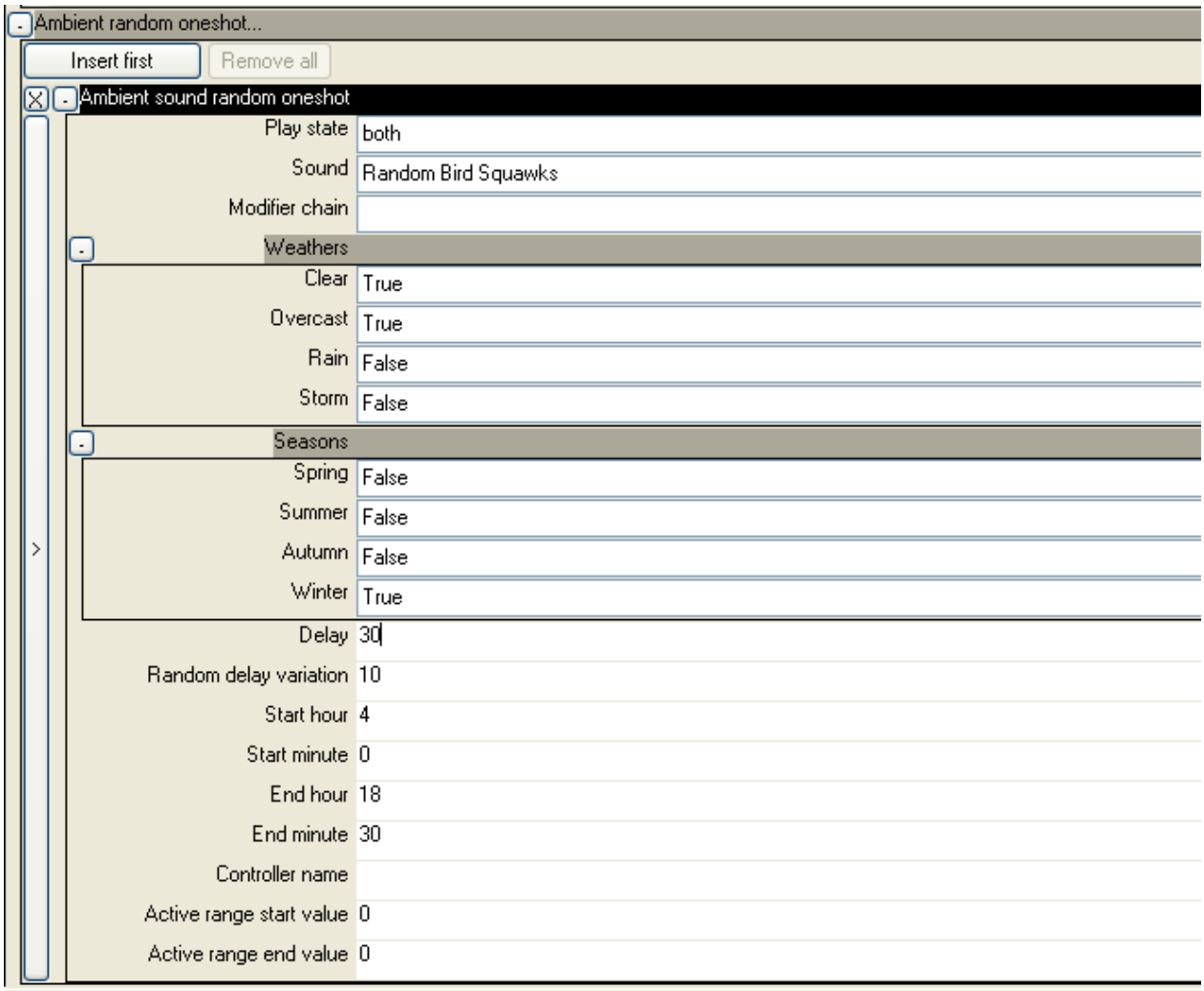
### 1.2.2 Ambient Random Oneshots

Ambient Random Oneshots work in pretty much exactly the same way as above, and should also have the **ListenerDistanceSquared** modifier chain applied if they are **Stereo** Sounds. Oneshots are more likely to be mono though, which is the case shown below. In the case of **Mono** sounds remember that the Attentuation Start and end distances as described in the sound section of the Audio Control are once again used. So the lesson here is:

## Mono sounds are a lot easier than Stereo Sounds!

So Ambient Random Oneshots will play random sounds to give you much more variation in your ambient sounds. Using random oneshots in combination with loops will give the most varied and memory efficient solution to ambient sound.

© Copyright Rail Simulator Developments 2007, all rights reserved



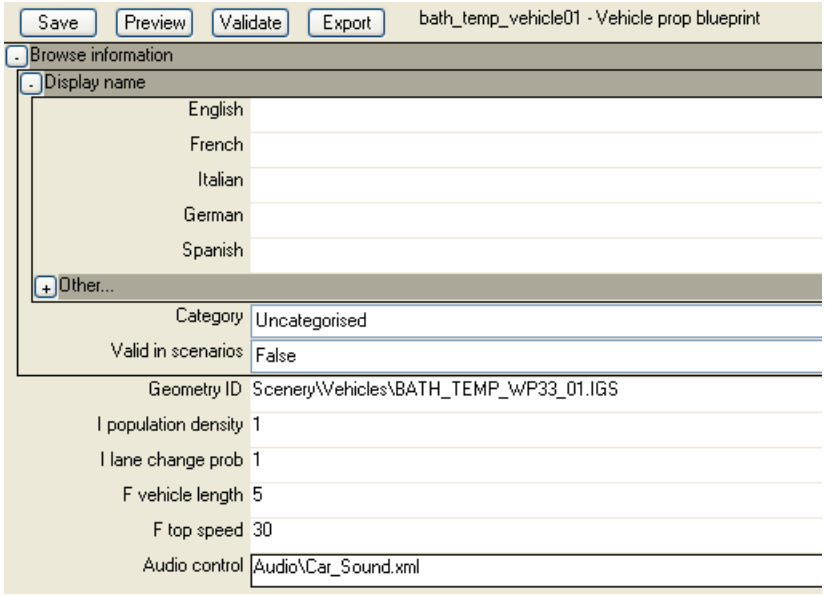

So this example is very similar to the last example with a couple of exceptions. A modifier chain is not required because this is a **MONO** sound and the attenuation of the volume with distance is handled automatically. Also at the bottom you'll notice we don't need a Controller name, the sounds will simply start playing once we are inside the Activation distance of **800**, as defined at the very top of the **Ambient Audio control**.

© Copyright Rail Simulator Developments 2007, all rights reserved

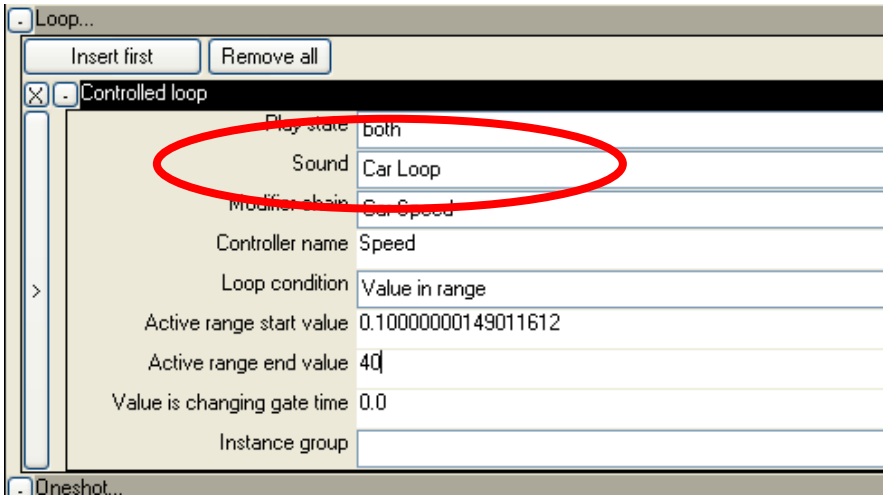
So the difference here is the **Delay** and **Random Delay Variation** values. These describe how often the random oneshots are triggered and heard. The **Delay** time is the basic time in seconds between triggers, so we will hear a sound every 30 seconds, however this also interacts with the **Random Delay Variation** of 10, which will add a random value between -10 -> + 10 to our delay value. The result of this is we will get a trigger every 20 -> 40 seconds. So it will not predictably trigger a sound every 30 seconds, which the user may begin to detect .

## 1.3 Road Sounds

Within a Vehicle Property Blueprint there is a field where you can specify an audio control, e.g.



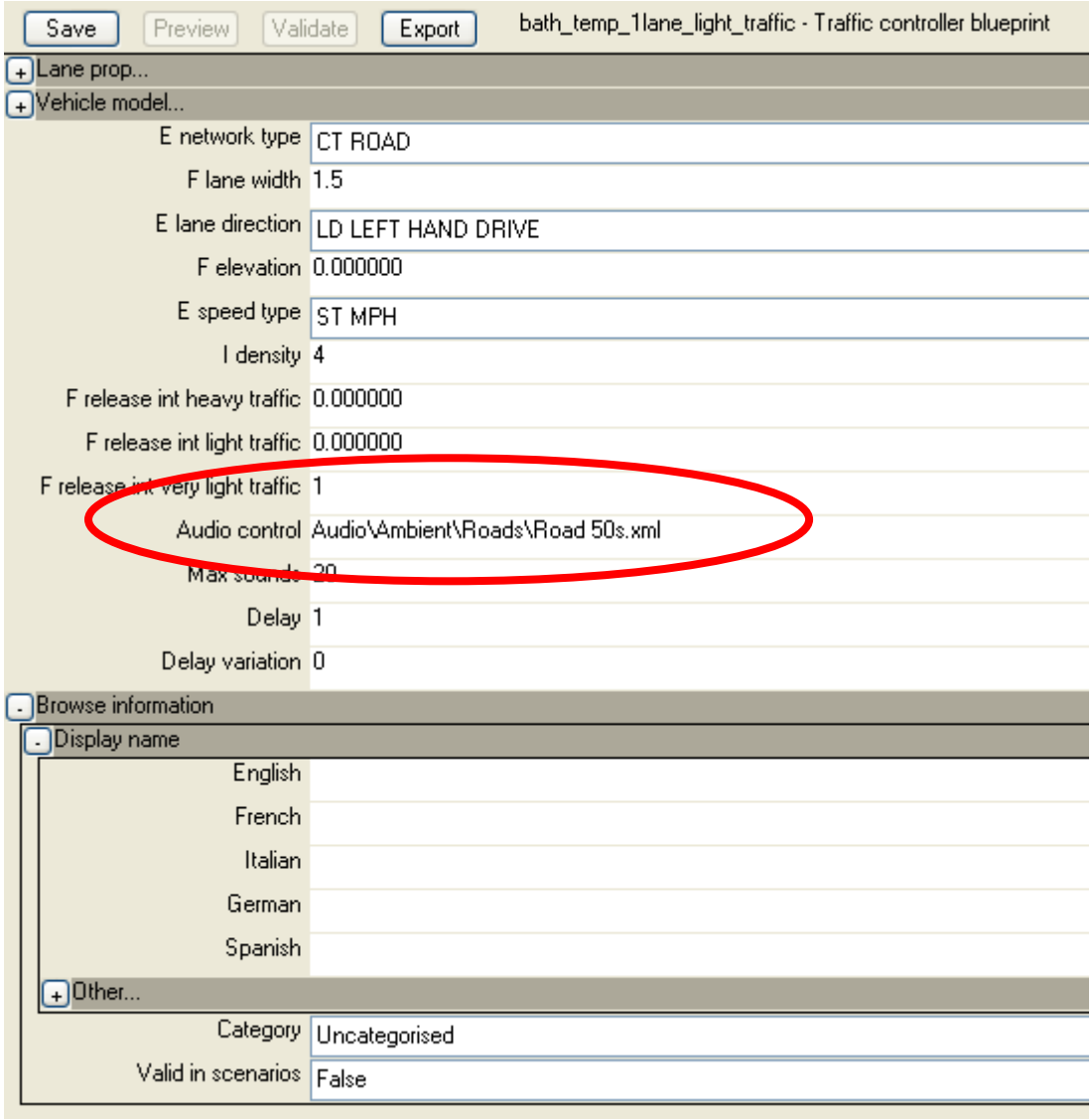

Which you can link to a **Generic Audio Control** with a car engine loop something like this




© Copyright Rail Simulator Developments 2007, all rights reserved

In this case **Speed** will return the speed of the car on the road, so you can use this value to control the volume and pitch of the car via a modifier chain.

There is also a section for an Audio Control in the Traffic controller blueprint. Here you can set an Audio Control which will provide a constant loop of background traffic for busy roads and also you specify the **Max Sounds** which will determine how many cars you can hear at once on a road, i.e. how many instances of the Audio control referenced in the **Vehicle property blueprint** are spawned. On very busy roads you could easily spawn hundreds of car sounds if you are not careful with this value and the users machine will quickly run out of sound channels.



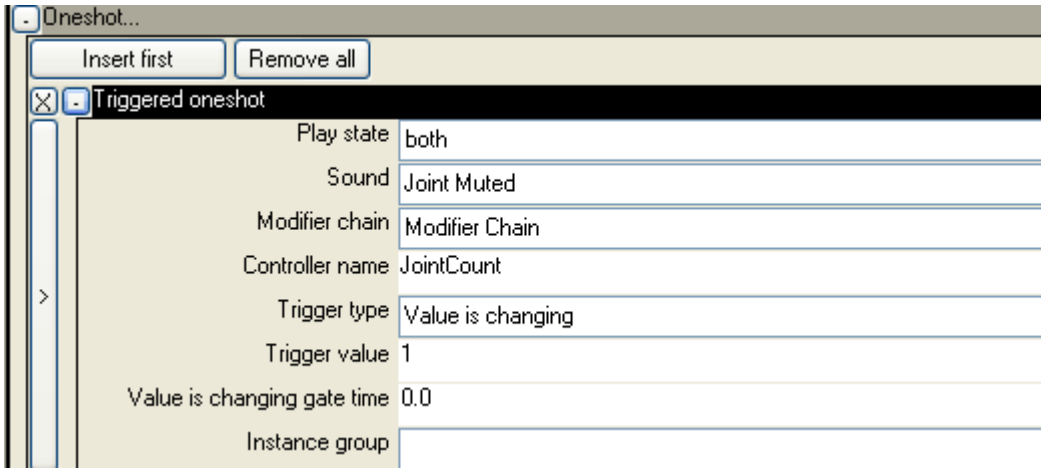

© Copyright Rail Simulator Developments 2007, all rights reserved

# 2 RailNetwork Sounds

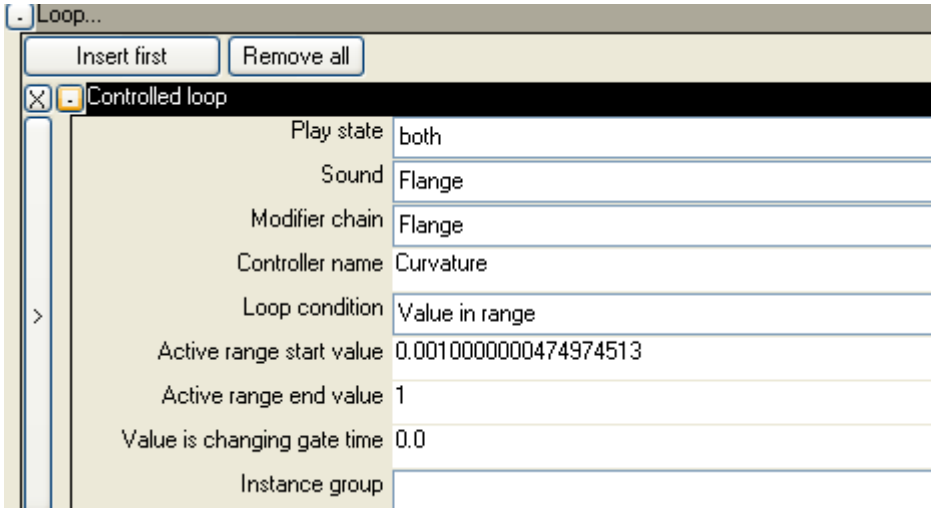
## 2.1 Bogies

The **Bogie Audio Control (Generic Audio Control)** is referenced in the Engine or Wagon Blueprint and allows you to add rail joints and flanging sounds.

So to add Rail joint sounds to an engine or wagon. Use the controller **JointCount** as below. The actual distance between joints is set up in the TrackBedRumble blueprint.




For flanging sounds use the **Curvature** controller like so:

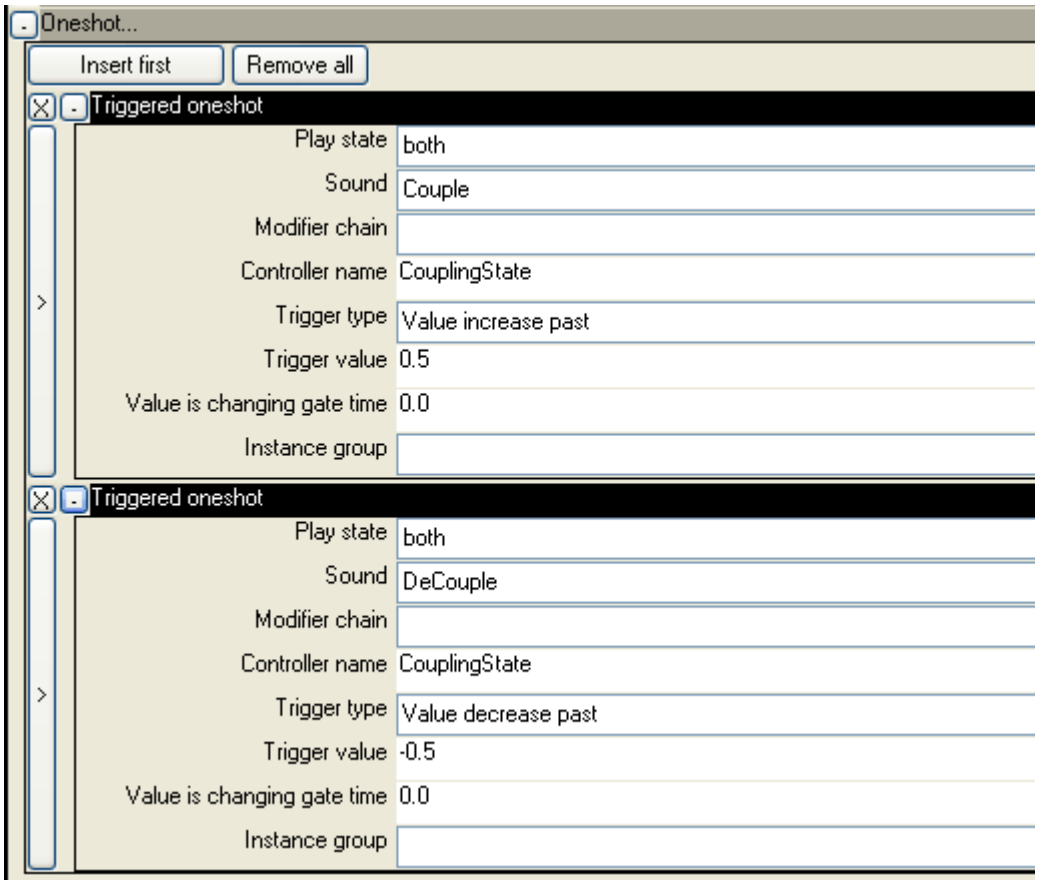



**Curvature** returns the value of the current radius of the track you are on. In the form 1/radius. So a track radius of 200 metres is returned as 0.005. Meaning that a curvature of zero is infinite radius, or in other words straight track. You can then use a modifier chain which is based on the consists **AbsoluteSpeed** and **Curvature** to control when the flange sound actually plays.

© Copyright Rail Simulator Developments 2007, all rights reserved

## 2.2 Coupling

The **Coupling Audio Control (Generic Audio Control)** is referenced in the Engine or Wagon Blueprint and allows you to add coupling or decoupling sounds.

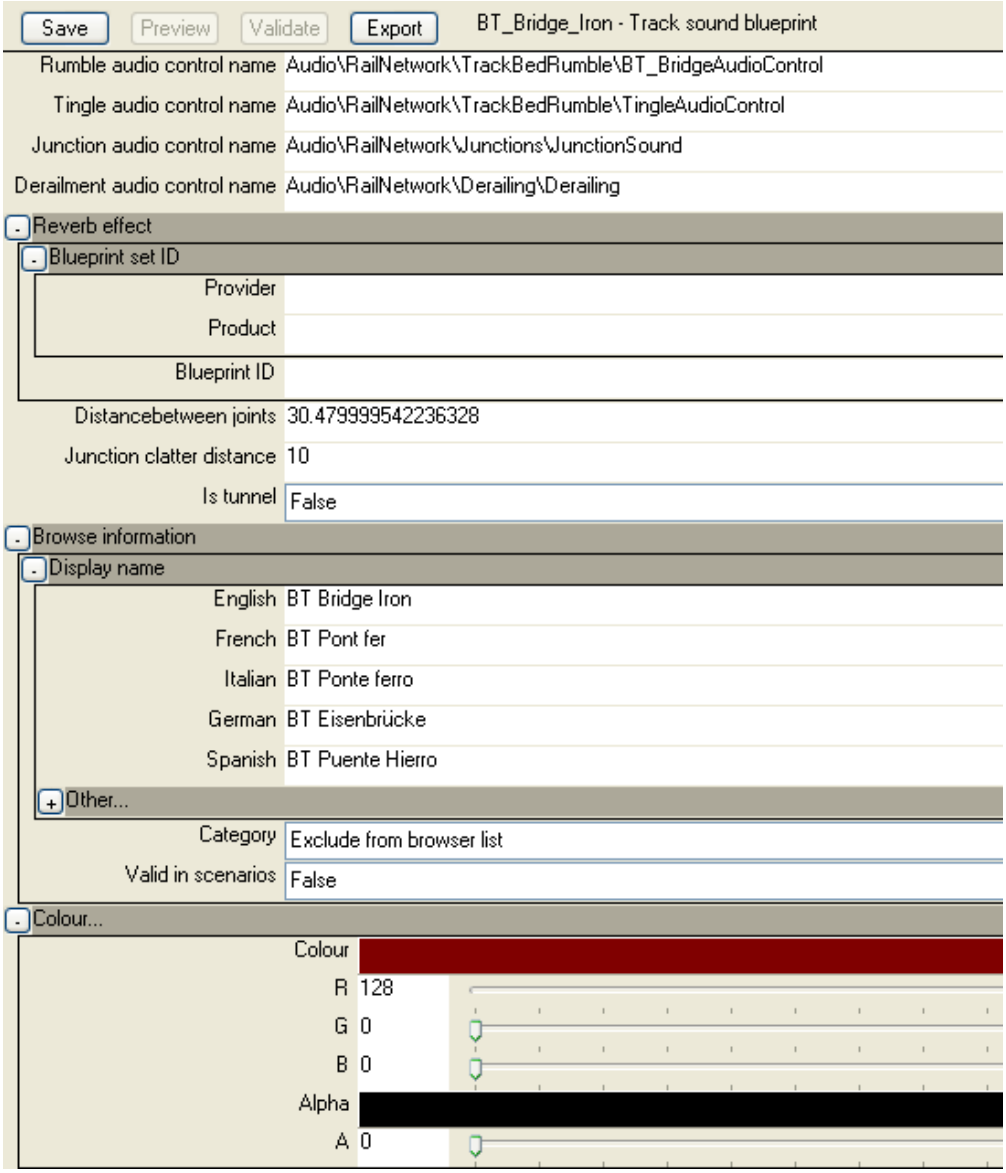



When **CouplingState** is '1' it is coupled, when it is set to '-1' it is decoupled. So you can you oneshots as above to trigger Couple and Decouple sounds.

© Copyright Rail Simulator Developments 2007, all rights reserved

## 2.3 TrackBedRumble

Here is an example of a typical TrackBedRumble or Track Sound Blueprint. In this case for a rumble sound you get when you pass over a bridge.




This Blueprint mainly houses references to specific audio controls which are described in following sections, **Rumble**, **Tingle**, **Junction**, **Reverb** and **Derailment**.

Ideally you need to create a new **Track Sound Blueprint** for each unique track type in the game. The minimum we suggest you need is one for external track and one for tunnel track.

© Copyright Rail Simulator Developments 2007, all rights reserved

**Distance between Joints** (metres) and **Junction clatter distance** determine how often and when the joint sounds specified in the **Bogies** audio control are triggered. So on a non-welded continuous type rail you should use a **Distance between Joints** of zero (no rail joints ever heard) or perhaps a very large value if there are the very occasional joints in the continuous rail (say every 10km). On a older steam route with welded rail you should use a **Distance between Joints** value specific to that route, the image above shows a joint distance of 30 which is relevant to Rail Simulators Bath-Templecombe route.

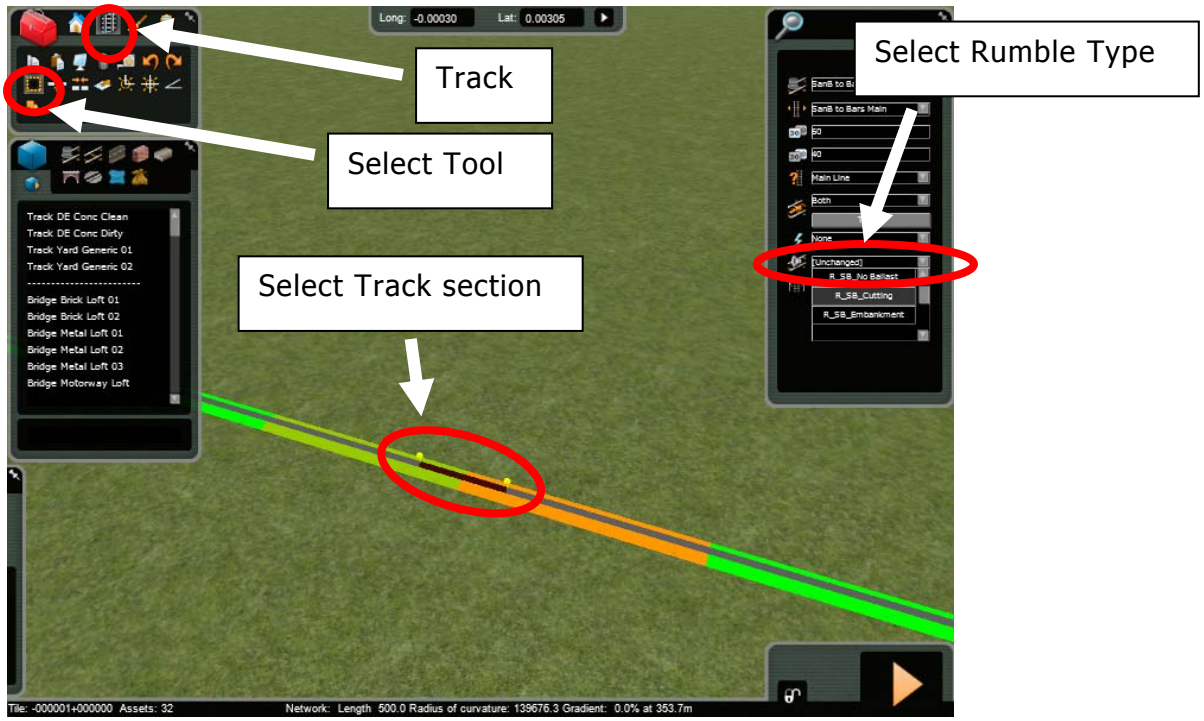
Category should be set to **Exclude from browser list** as you will never want to place this blueprint directly in the editor.

Colour determines how the particular rumble type will look in the editor, to lay down different rumble types on a piece of track you need to go to…

Editor -> Track -> Select tool

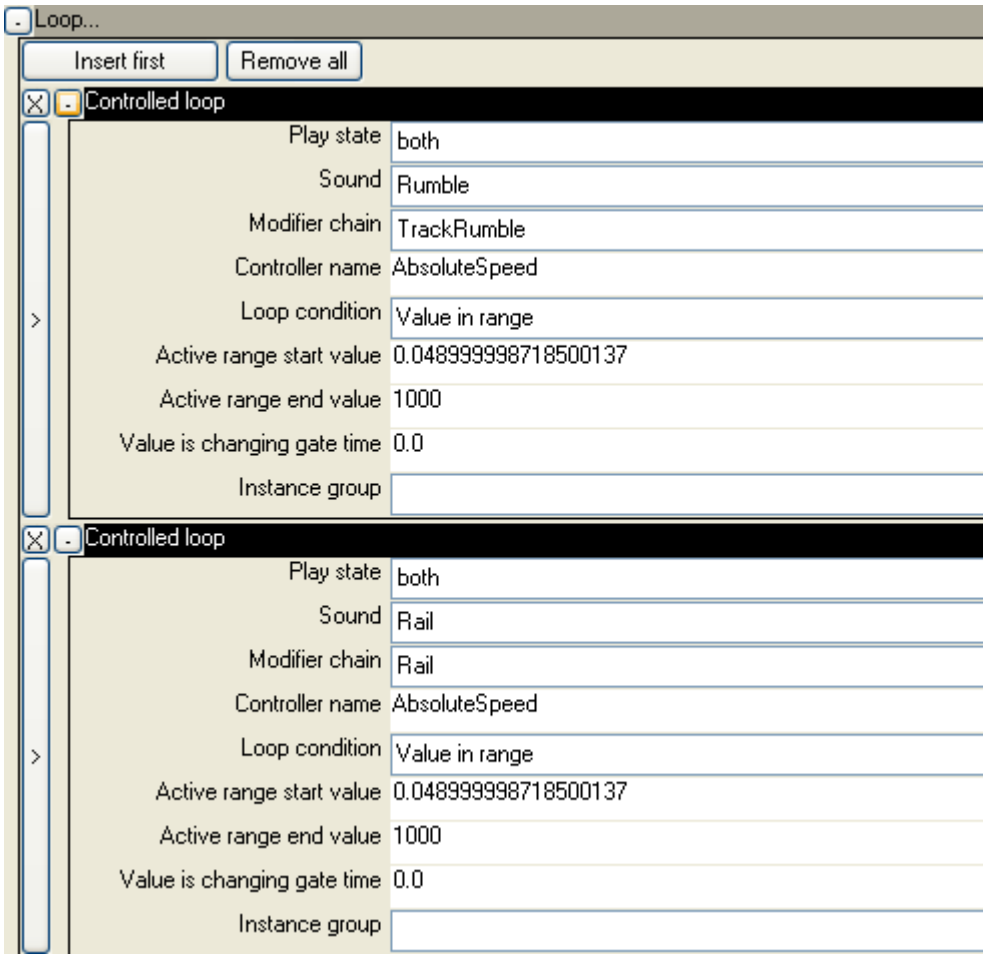
Select a piece of track.

Open the right hand panel and select the relevant rumble type. The **Track Sound Blueprints** must be referenced in the **Track Blueprint** for them to appear in this list. It is also very important that the **Default Rumble Blueprint** is set in the **Track Blueprint** before laying **ANY** track in the editor. Otherwise you will need to set rumbles for 90% of the laid track manually. Which is a long task.




© Copyright Rail Simulator Developments 2007, all rights reserved

Here is a typical section of a **Rumble Audio Control (Generic Audio Control)**

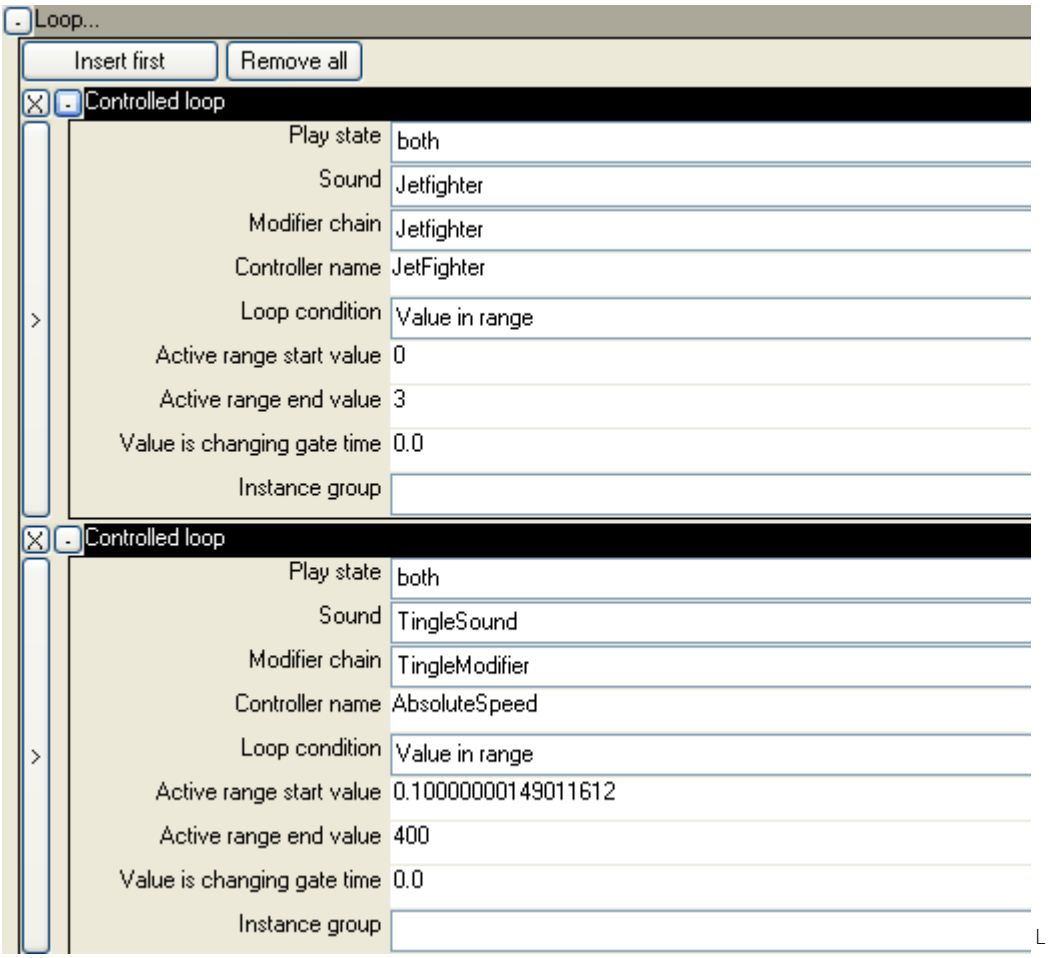



The positioning of the rumble sound is handled automatically by the sound engine. The Position of the track rumble is snapped to the track, within the length of the player consist and at the nearest point to the 'screen'.

As you can see above, Controlled loops are the order of the day here. **AbsoluteSpeed** is the controller which you should use to control the volume of the rumble based upon speed, via a curve and modifier chain. The recommended start value is 0.05. This means the rumble will not play whilst the train is stationary and will negate any slight 'flutter' of the **AbsoluteSpeed** value whilst the train is stationary.

© Copyright Rail Simulator Developments 2007, all rights reserved

Here is a typical section of a **Tingle Audio Control (Generic Audio Control)**



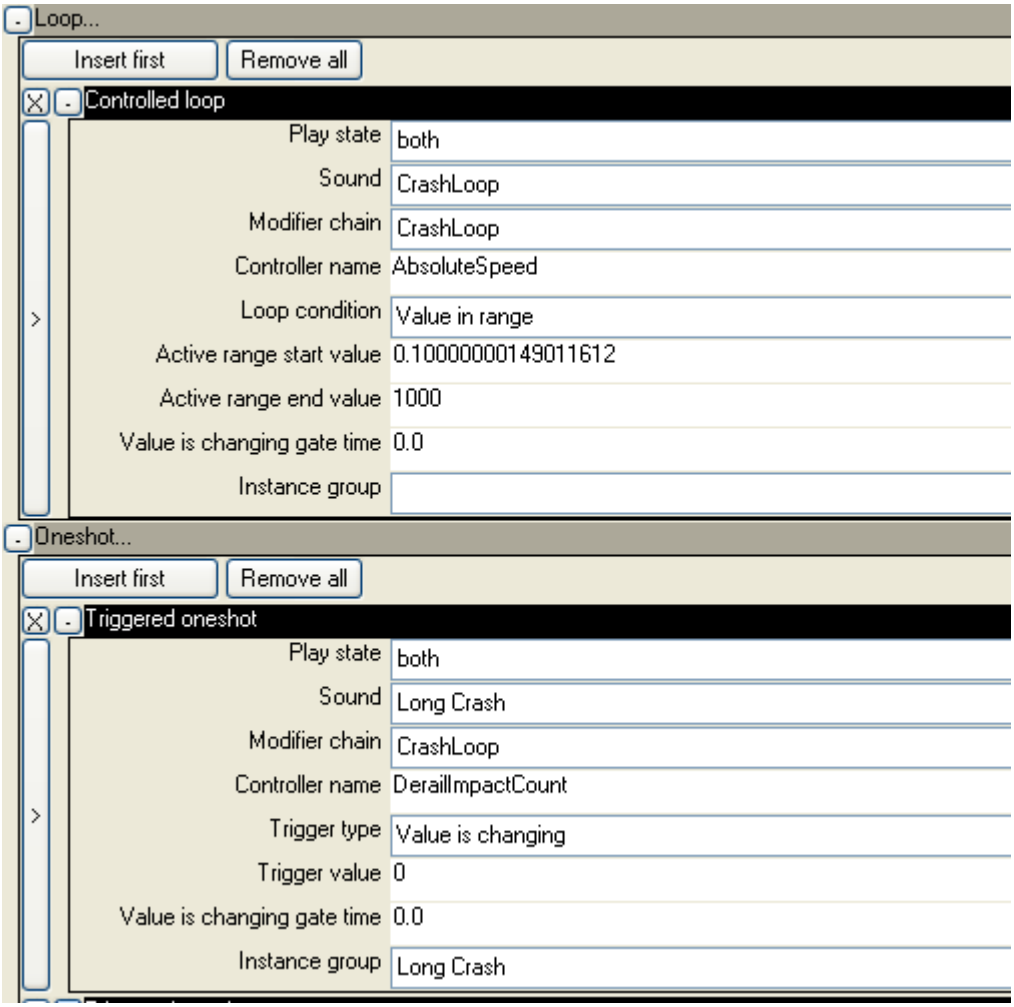

The tingle is used to control two effects in Rail Simulator, although a whole range of potential effects are possible.

The tingle effect is the sound of the rail vibrating as a train is approaching. This is controlled again via **AbsoluteSpeed** but also via the parameter **DistanceToConsist** which is the distance to the nearest point of the consist in question.

The **JetFighter** controller is used to playback the sound of rushing air as a train passes.
It is essentially identical to the **DistanceToConsist** controller except when in use on a player consist it will return a value of zero when in any camera mode where you are attached to the consist.

© Copyright Rail Simulator Developments 2007, all rights reserved

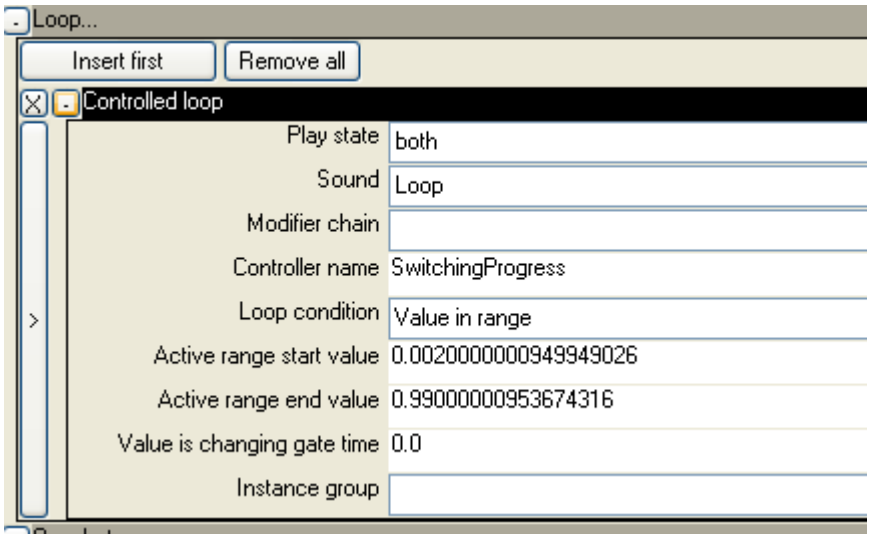
## 2.4 Derailing




The **Derailing Audio control** is a **Generic Audio Control** begins working when the consist derails. At this point the **AbsoluteSpeed** value of the consist can be used to trigger a grinding or rumble loop of the train moving across the ground. The **DeraillmpactCount** controller will increment everytime rolling stock hits another object, it is also automatically incremented by 1 at the point the train derails allowing for immediate impact effects.

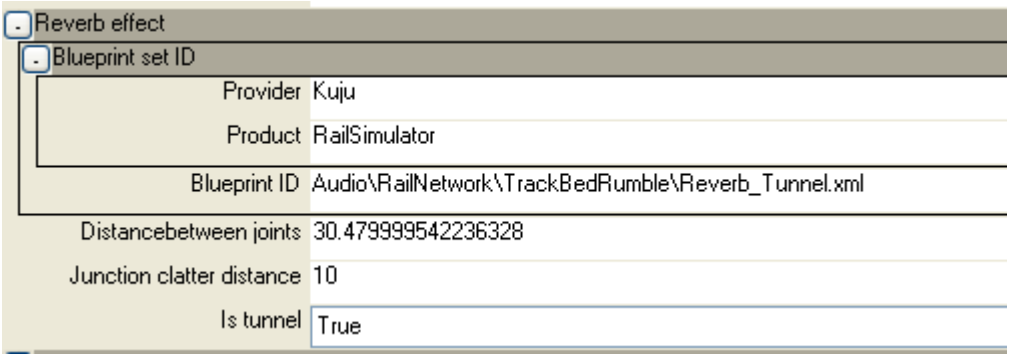
© Copyright Rail Simulator Developments 2007, all rights reserved

## 2.5 Junctions




Junction sounds can be added using the **SwitchingProgress** controller which changes smoothly between 0-1

## 2.6 Reverb




**Reverb effect blueprints** are referenced in the **Track sound blueprint.** This effect requires an EAX enabled card. Below is an example of typical settings for a Tunnel style reverb.

© Copyright Rail Simulator Developments 2007, all rights reserved

Save | Preview | Validate | Export | Reverb_Tunnel - Reverb effect blueprint

| | | |
|---|---|---|
| Send level | -200 | MILLIBELLS |
| Environment size | 2 | METRES |
| Environment diffusion | 1 | |
| Room | 0 | MILLIBELLS |
| Room HF | -1000 | MILLIBELLS |
| Room LF | -100 | MILLIBELLS |
| Decay time | 8 | SECONDS |
| Decay HF ratio | 0.89999997615814209 | |
| Decay LF ratio | 1 | |
| Reflections | -6000 | MILLIBELLS |
| Reflections delay | 0.0070000002160668373 | SECONDS |
| Reflections pan x | 0 | |
| Reflections pan y | 0 | |
| Reflections pan z | 0 | |
| Reverb | 200 | MILLIBELLS |
| Reverb delay | 0.010999999940395355 | SECONDS |
| Reverb pan x | 0 | |
| Reverb pan y | 0 | |
| Reverb pan z | 0 | |
| Echo time | 0.10000000149011612 | SECONDS |
| Echo depth | 0 | |
| Modulation time | 0.25 | SECONDS |
| Modulation depth | 0 | |
| Air absorption HF | -5 | MILLIBELLS_PER_METRE |
| HF Reference | 5000 | HERTZ |
| LF Reference | 250 | HERTZ |
| Room roll off factor | 0 | |
| Decay time scale | False | |
| Reflections scale | False | |
| Reflections delay scale | False | |
| Reverb scale | False | |
| Reverb delay scale | False | |
| Echo time scale | False | |
| Modulation time scale | False | |
| Reverb decay HF limit | False | |

## 2.7 Platforms

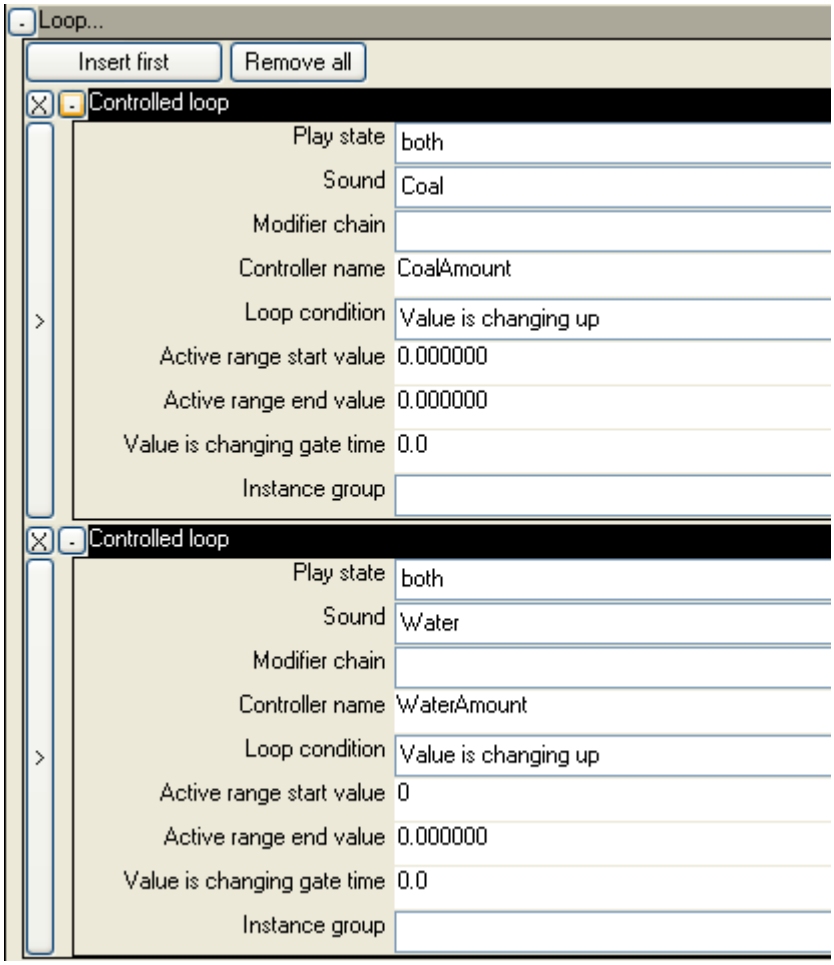
Platforms work in a similar way to road sounds. If you attach a generic blueprint as a child to a platform blueprint you can use the controller **NumberOfPeopleOnPlatform** to control the level of an overall ambience loop or also oneshots such as coughs, footsteps, mobile phone rings etc.

## 2.8 Signals

In order to make sounds when a mechanicl signal moves the signal script must create a controller such as **SignalProgress** which would work in a similar way to the junction controller.

© Copyright Rail Simulator Developments 2007, all rights reserved

## 2.9 TransferPoints




There are three controllers which are used to create refuelling sounds.

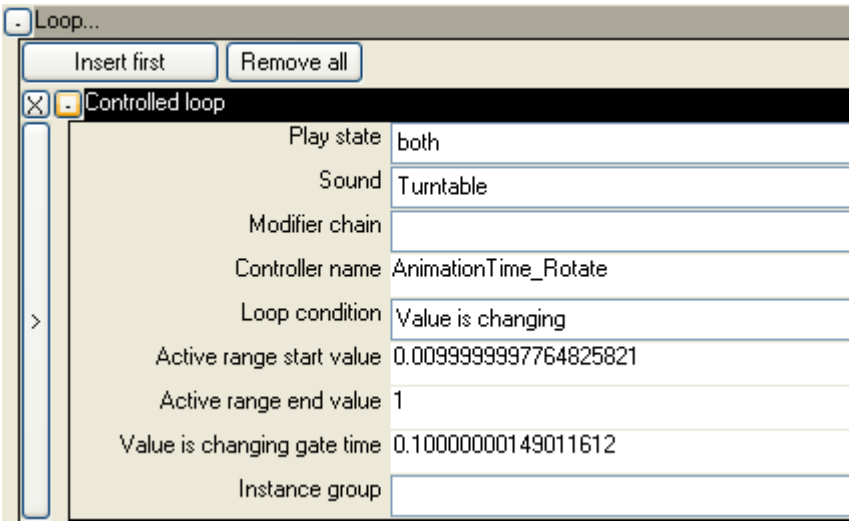
**CoalAmount**
**WaterAmount**
**DieselAmount**

These refuelling sounds will actually be used in the relevant rolling stock audio control.
You can then trigger sounds to play back whilst in the condition 'Value is changing up', which naturally only happens while the rolling stock is refuelling/loading.

© Copyright Rail Simulator Developments 2007, all rights reserved

### 2.10 Traversers + Turntables

These generic audio controls are added as a child to the relevant track furniture object.




Traversers and Turntables work simply with a controller that is based on the name of the animation which control them. So in this case we are using a controller called **AnimationTime_Rotate** as the Turntable object is animated using a file called **Rotate.ia**

If the animation was called for example **Turntable.ia** then a controller would automatically be created called **AnimationTime_Turntable**

**AnimationTime** controllers are in seconds. If you just want to trigger a loop while an animation is moving (as in the case of a turntable like above) then you can just use 'Value is changing'. If you want to trigger sounds only at a particular point in the loop then just use the timing in seconds of where it occurs in the animation, sometimes it's easiest to get the animator to provide you with relevant timings.

# 3 Locomotives

Locomotive audio is certainly the most important area of sound design in Rail Simulator, if you've got this in the document then you will already have picked up the necessary skills to

© Copyright Rail Simulator Developments 2007, all rights reserved

start creating engine and wagon audio controls. This is not the point to dive straight in though!

Below are lists of relevant controllers for use in the different engine types. All rolling stock will also inherit any controllers of the **AnimationTime_** type as described in the **Turntable** section above and also any 'controls' which have been set up in the **Engine Blueprint**.

### 3.1 Diesel Engines

**RPM** – This must be added as a control to the **Engine Blueprint** although requires no animation or visible element to work. It is the primary control over sound parameters for a diesel train. You will also need to tweak the RPM change per second, and the engine startup and shutdown times in the **engine simulation** file to fine tune various aspects of the train audio.
**RPMDelta** – This must be added as a control to the **Engine Blueprint** although requires no animation or visible element to work. This value gives you the rate of change of the RPM in order to trigger surges when the RPM changes above a certain rate. The value is from -1 -> 1
**CompressorState** – Changes from 0 -> 1 when the compressor engages (controlled by simulation)
**ImpactCount** – This value increments everytime the train collides with something but does not derail.
**DieselAmount** – The current fuel amount.
**DistanceTravelled** – The current distance travelled in metres since the scenario was loaded.
**BrakeAmount** – The total combined braking force on the Consist 0 -> 1
**TrainBrakeAmount** – As BrakeAmount but for the Train Brake only.
**LocoBrakeAmount -** As BrakeAmount but for the Loco Brake only.
**ControlType** – 0=simple, 1=intermediate, 2= Expert
**AbsoluteSpeed** – The speed in **metres per second.**
**WheelSlipAmount** – 0 = Train Stationary, 1=Normal running (no slip), 3=Full wheelslip.
**Ammeter** – Useful for triggering sounds when making field diverts or for a general change in sound as the tractive effort increases.

© Copyright Rail Simulator Developments 2007, all rights reserved

## 3.2 Electric Engines

**Current** – This must be added as a control to the **Engine Blueprint** although requires no animation or visible element to work. This is the key value to base the sound of an electric train upon.
**CompressorState** – Changes from 0 -> 1 when the compressor engages (controlled by simulation)
**ImpactCount** – This value increments everytime the train collides with something but does not derail.
**DistanceTravelled** – The current distance travelled in metres since the scenario was loaded.
**BrakeAmount** – The total combined braking force on the Consist 0 -> 1
**TrainBrakeAmount** – As BrakeAmount but for the Train Brake only.
**LocoBrakeAmount -** As BrakeAmount but for the Loco Brake only.
**ControlType** – 0=simple, 1=intermediate, 2= Expert
**AbsoluteSpeed** – The speed in **metres per second.**
**WheelSlipAmount** – 0 = Train Stationary, 1=Normal running (no slip), 3=Full wheelslip.

## 3.3 Steam Engines

**Engine**

**ImpactCount** – This value increments everytime the train collides with something but does not derail.
**DistanceTravelled** – The current distance travelled in metres since the scenario was loaded.
**BrakeAmount** – The total combined braking force on the Consist 0 -> 1
**TrainBrakeAmount** – As BrakeAmount but for the Train Brake only.
**LocoBrakeAmount -** As BrakeAmount but for the Loco Brake only.
**ControlType** – 0=simple, 1=intermediate, 2= Expert
**AbsoluteSpeed** – The speed in **metres per second.**
**WheelSlipAmount** – 0 = Train Stationary, 1=Normal running (no slip), 3=Full wheelslip.
**Tender includes all above parameters plus:**

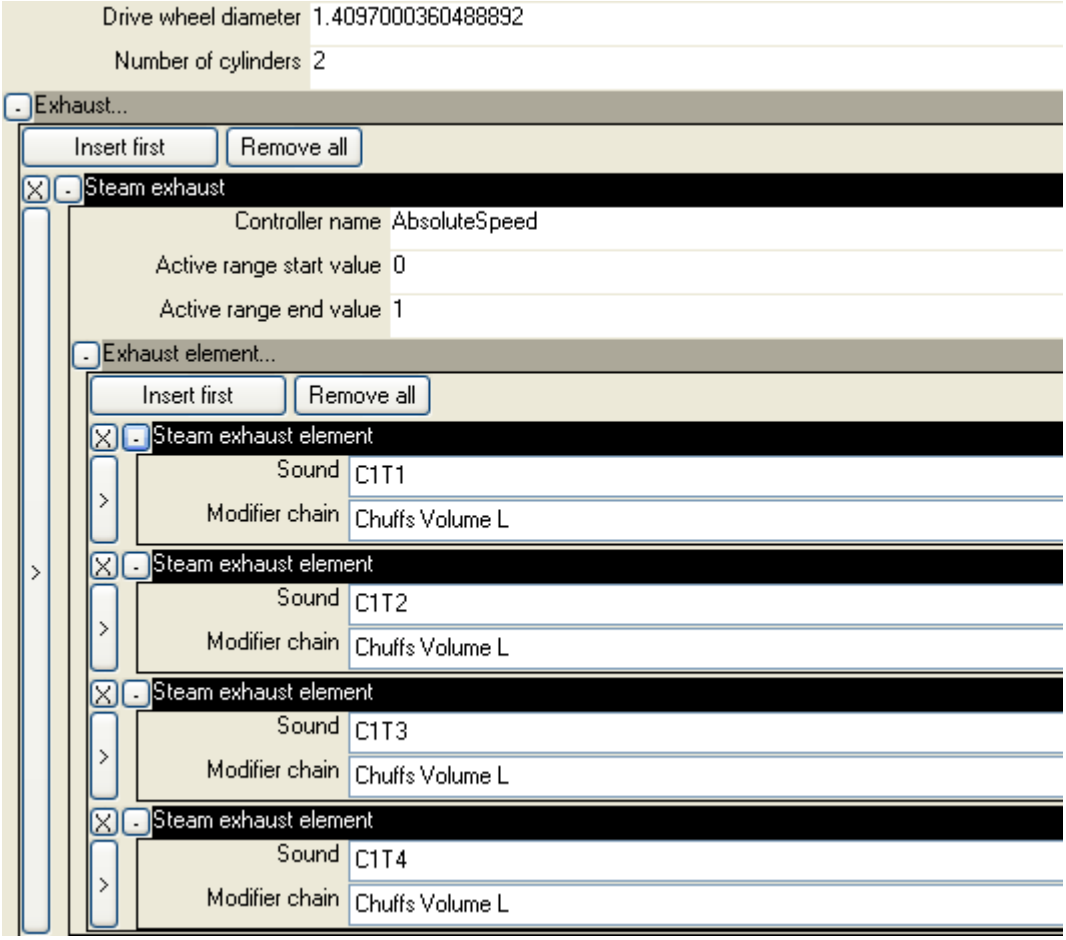
**WaterAmount** – The current fuel amount.
**CoalAmount** - the current fuel amount.

© Copyright Rail Simulator Developments 2007, all rights reserved

The Steam Audio control also contains a unique section which is used to trigger steam exhausts (chuffs) and any regular mechanical type sounds which are based on the rotation of the main driving wheel (see image below)

It is vital that the two parameters **Drive wheel diameter** and **Number of Cylinders** are set correctly in order to have the correct number of chuffs per drive wheel rotation.

The Steam Audio control has a unique **Exhaust** section which is essentially a oneshot controller.




To add chuff sounds insert a **Steam Exhaust** in the **Exhaust** section. Use **AbsoluteSpeed** as the controller with a valid start and end value. This way you can use various sets of chuffs of shorter lengths for different train speeds.

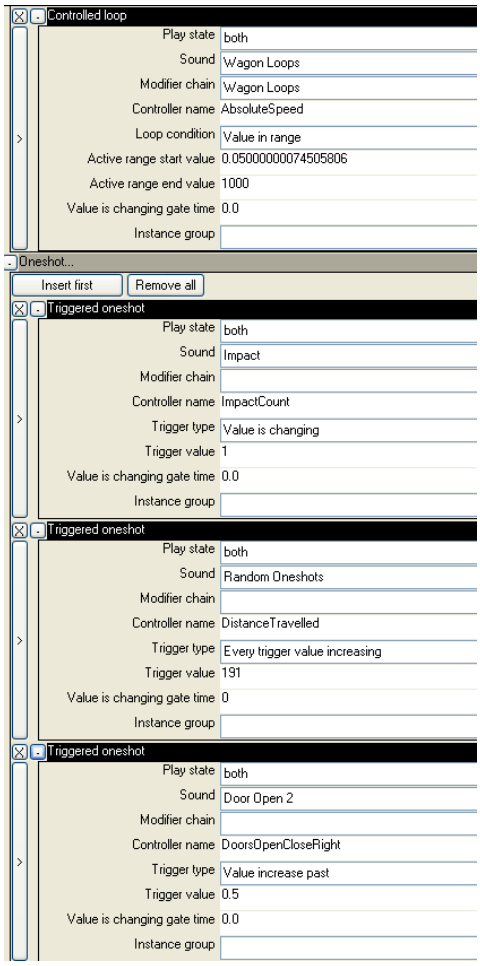
If you add one **Steam Exhaust Element** called **1** then everytime a chuff is triggered you will trigger **1, 1, 1, 1, 1, 1** etc

If you add three **Steam Exhaust Element** called **1, 2** and **3** then everytime a chuff is triggered you will trigger a repeating sequence of **1, 2, 3, 1, 2, 3, 1, 2, 3**. Of course each sound referenced in here could also be a **random oneshot** to add variation.

© Copyright Rail Simulator Developments 2007, all rights reserved

## 3.4 Wagons

### 3.4.1 Wagons




Above are some examples of typical controllers used on wagons and passenger carriages.

**DistanceTravelled** – The current distance travelled in metres since the scenario was loaded. It can be used as above with a oneshot using the 'Every Trigger value increasing' to trigger a sound every 150 metres or whatever cycle you prefer.
**AbsoluteSpeed** – The speed in **metres per second.**
**DoorsOpenCloseRight** and **DoorsOpenCloseLeft** – Used to trigger sounds when passenger train doors are opened and closed.

© Copyright Rail Simulator Developments 2007, all rights reserved

### 3.4.2 Passenger Carriage Occlusion

| | | |
|---|---|---|
| Occlusion | -2400 | MILLIBELLS |
| Occlusion LF ratio | 0.0099999997764825821 | |
| Occlusion room ratio | 0.5 | |
| Occlusion direct ratio | 1 | |

This is used to control how much the sound is occluded (or muffled) when you are inside the passenger carriage, basically it is used to decrease the amount of high frequency content. It won't effect any sounds which are marked as **INSIDE** in the passenger/wagon blueprint controlled loops and oneshots.

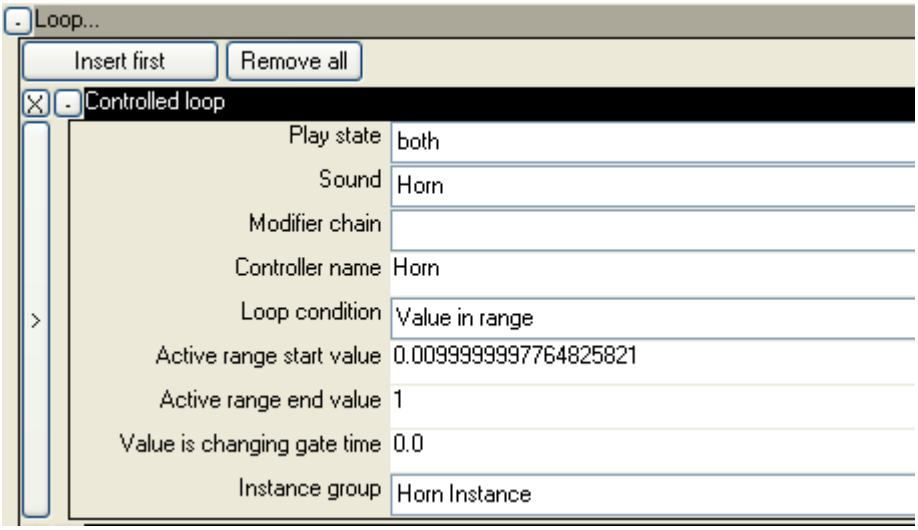
Refer to the document **EAX4 Sound Designer's Guide (A4).pdf** for descriptions of these values.

## 3.5 Cabs

### 3.5.1 Cabs

You use the **cab audio control** to trigger all sounds based on the levers, buttons, switches etc moving in the cab. The controllers for this are all inherited from the controls set up in the **Engine Blueprint**, Regulator, Reverser, Brakes etc.

### 3.5.2 Horns




It is best to do the horn/whistle as a separate sound child of the engine for two reasons:

1) If it was part of the cab audio control you would not hear it outside of the cab (the same goes for windscreen wipers)

2) you are likely to want to hear the horn further away than the engine sounds. I.e. set a higher activation distance and attenuation settings.

© Copyright Rail Simulator Developments 2007, all rights reserved

The horn relies on a control being created, which in this case was called simply Horn in the **Engine Blueprint**.

### 3.5.3 Occlusion

| | | |
|---|---|---|
| Occlusion | -2400 | MILLIBELLS |
| Occlusion LF ratio | 0.0099999997764825821 | |
| Occlusion room ratio | 0.5 | |
| Occlusion direct ratio | 1 | |

This is used to control how much the sound is occluded (or muffled) when you are inside the engine cab, basically it is used to decrease the amount of high frequency content. It won't effect any sounds which are marked as **INSIDE** in the cab blueprint controlled loops and oneshots.

Refer to the document **EAX4 Sound Designer's Guide (A4).pdf** for descriptions of these values.

© Copyright Rail Simulator Developments 2007, all rights reserved

# 4 Animated Scenery

Animated scenery works exactly the same as the turntable described earlier using the controller **AnimationTime_*AnimationName***. (do not include the .ia extension)

These are objects such as the Digger and Forklift in Rail Simulator which were placed directly in the world. The Audio blueprint must be added as a child to the object.

© Copyright Rail Simulator Developments 2007, all rights reserved

# 5 Weather

The **Weather Audio Control** is referenced directly from the **Weather Pattern Blueprint**.

There are 5 controllers with which to control weather sounds:

**Windspeed**
**RainDensity**
**HailDensity**
**SnowDensity**
**SleetDensity**

These are all based on the same units as set up in the **Weather Pattern Blueprint**. If you want the rain sound to change loop as the RainDensity increases then you need to create various modifier curves to crossfade between different loops.

© Copyright Rail Simulator Developments 2007, all rights reserved